

Parametric Recognizability Over the Integers

ANONYMOUS AUTHOR(S)

Büchi arithmetic on a fixed integer $k \geq 2$ is the first-order theory of the structure $(\mathbb{Z}; 0, 1, +, V_k, \leq)$, where the binary relation $V_k(x, y)$ is true if and only if y is the greatest power of k dividing x . By the Büchi-Bruyère theorem, this theory completely describes the sets of integers that are recognizable by finite automata when encoded in the k -ary positional number system.

Motivated by recent advances in symbolic automata, in this paper we study expressivity and decidability problems for Büchi arithmetic with a *parametric positional encoding*, where k is treated as a variable rather than a fixed constant. We show that this *parametric* Büchi arithmetic provides a logical characterization of a class of *parametric symbolic automata* over Linear Integer Arithmetic (LIA), thus extending the Büchi-Bruyère theorem to the parametric setting. As a corollary, we obtain that, for every first-order sentence φ_t (parametric in t) in the language of $(\mathbb{Z}; 0, 1, +, V_t, \leq)$, the set of bases $k \geq 2$ where φ_k is true (in $\mathbb{Z}$) is ultimately periodic.

At a technical level, the original Büchi-Bruyère theorem translates finite automata to the $\exists^*\forall^*$ fragment of Büchi arithmetic, while our proof translates parametric symbolic automata to the $\exists^*\forall^*\exists^*$ fragment of parametric Büchi arithmetic. We trace this discrepancy in the number of quantifier alternations to the use of the map $x \mapsto k \cdot x$ multiplying a natural number by the fixed base k , whose parametric counterpart $x \mapsto t \cdot x$ is not definable in parametric Büchi arithmetic. Indeed, we show that given a $\exists^*\forall^*$ sentence φ_t in the language of $(\mathbb{Z}; 0, 1, +, x \mapsto t \cdot x, V_t, \leq)$, the existence of a base $k \geq 2$ such that φ_k is true is an undecidable problem.

Finally, we provide a prototype implementation of both parametric and standard symbolic automata, and experiments on problems that ask to decide whether a formula is satisfied *for all (or for some) bases* $k \geq 2$ in an input segment $[\alpha.. \beta]$. These experiments indicate that parametric symbolic automata are more efficient when the range of bases is large, or so is α .

Additional Key Words and Phrases: Symbolic automata; Büchi arithmetic; Decidability; SMT-solving

1 INTRODUCTION

Establishing connections between computational models and arithmetic theories is a classic topic in theoretical computer science. The focus of this paper is related to “limitations” of a fundamental theorem of this kind, established by Büchi and Bruyère in [Bruyère 1985; Büchi 1960]. For a fixed integer $k \geq 2$, consider the first-order theory of the structure $(\mathbb{N}; 0, 1, +, V_k, \leq)$, which extends Linear Integer Arithmetic (LIA, also known as Presburger arithmetic) with a binary predicate V_k interpreted as follows: $V_k(x, y)$ is true whenever y is the largest power of k dividing x . For example, $V_2(14) = 2$, $V_2(15) = 1$, and $V_2(16) = 16$. The Büchi-Bruyère theorem tells us that the expressiveness of this arithmetic theory coincides to that of regular languages:

THEOREM 1.1 (BÜCHI-BRUYÈRE THEOREM [BRUYÈRE ET AL. 1994]). *Fix an integer $k \geq 2$. Then, every set $S \subseteq \mathbb{N}^n$ is recognizable by a finite automaton over the alphabet $\{0, \dots, k-1\}^n$ if and only if S is first-order definable in the structure $(\mathbb{N}; 0, 1, +, V_k, \leq)$.*

Here, non-negative integers can be encoded in the most- or least-significant digit first notation (*msdf* and *lsdf*, respectively) as words $a_{\ell-1} \dots a_1 a_0$ in the language $\{0, 1, \dots, k-1\}^+$. In the case of *msbf*, this string defines the integer $a_{\ell-1} \cdot k^{\ell-1} + \dots + a_1 \cdot k + a_0$. We say that $S \subseteq \mathbb{N}$ is *recognizable* by an automaton $\mathcal{A}$ if it accepts exactly *all* strings representing elements of S . By reading k -ary expansions of integers synchronously, the definition is generalized on dimensions $n \geq 1$ (see formal definition in Section 2.5). A set $S \subseteq \mathbb{N}^n$ is (first-order) *definable* in the structure $(\mathbb{N}; 0, 1, +, V_k, \leq)$ if there is a formula φ in the first-order language of this structure with n free variables such that $\mathbf{a} \in S$ if and only if $\varphi(\mathbf{a})$ is true. Then, Theorem 1.1 states that *recognizability* by finite automata

corresponds to *definability* in $(\mathbb{N}; 0, 1, +, V_k, \leq)$. This result can be naturally extended to work over the integers [Boigelot 2021]. In this case, integers are represented in the k 's complement encoding, i.e., as words $a_\ell \dots a_0$ in the language $\{0, k-1\} \cdot \{0, 1, \dots, k-1\}^*$. When $a_\ell = 0$, this string defines $a_{\ell-1} \cdot k^{\ell-1} + \dots + a_1 \cdot k + a_0$. Otherwise, if $a_\ell = k-1$, it encodes $-k^\ell + a_{\ell-1} \cdot k^{\ell-1} + \dots + a_1 \cdot k + a_0$. For example, '9877' represents in the 10's complement encoding the integer $-123 = -1000 + 877$.

EXAMPLE 1.1. The two finite automata below recognize binary (Figure 1a) and decimal (Figure 1b) strings representing the integers x that satisfy the formula $V_k(x, x) \wedge \exists y : x = 3 \cdot y + 1$, i.e., powers of k (with $k = 2$ and $k = 10$, respectively) congruent to 1 modulo 3.

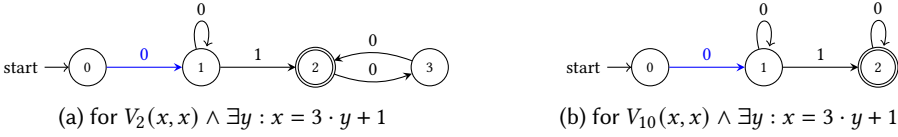


Fig. 1. Automata in base 2 and base 10 in signed msbf encoding.

For example, the automaton from Figure 1a accepts words '01', '0100', and '010000' representing the integers 1, 4, and 16, but does not accept any word with odd number of zeros after '1'. The automaton from Figure 1b accepts all powers of 10: they are always congruent to 1 modulo 3. The first transition (in blue) decides the sign of the integer. Transitions to sink states are omitted. ◀

1.1 Applications of Büchi-Bruyère theorem

The proof of Theorem 1.1 is constructive, and shows the decidability of the first-order theory of the structure $(\mathbb{Z}; 0, 1, +, V_k, \leq)$, which we henceforth call *Büchi arithmetic (on base k)*. This result gave raise to the study of *automatic structures* (the structures that can be represented by automata) [Grädel 2020; Hodgson 1976; Khoussainov and Nerode 1995], and of *numeration systems* admitting automatic representations [Bruyère and Hansel 1997; Hieronymi and Terry 2018; Shallit 1994].

In the context of automatic structures, Büchi arithmetic is bi-interpretable with the weak second-order theory of one successor (WS1S): alongside the translation from Büchi arithmetic to automata, in his pioneering paper [Büchi 1960] Büchi establishes translations between automata and WS1S in both directions. WS1S has found its applications in formal verification and is supported by several solvers [D'Antoni and Veanes 2017; Elgaard et al. 1998; Fiedor et al. 2017, 2019]. These applications extend to bit-vector analysis, since basic bitwise operations can easily be represented by finite automata and in Büchi arithmetic; see [Bhat et al. 2025; Schuele and Schneider 2006]. Büchi arithmetic is also related to the linear theory of p -adic numbers with divisibility $(\mathbb{Q}_p; 0, 1, +, |_p, =)$ [Hodgson 1976], where p is a fixed prime number; we touch upon this connection in Section 1.2.

In the context of numeration systems, the Büchi-Bruyère theorem has found important applications in word combinatorics, which studies periodicity and the (non-)existence of patterns in sequences of (infinite) words [Allouche and Shallit 2003]. Automata-based theorem provers such as Walnut [Mousavi 2016; Shallit 2022] and Pecan [Hieronymi et al. 2024; Oei et al. 2021] are actively used to prove new results in combinatorics on words (now in the hundreds), and to re-prove old ones—sometimes spotting and correcting errors; see, e.g., [Shallit 2022, Section 9.11.6].

Another consequence of Theorem 1.1 is that LIA and the classic integer linear programming feasibility problem (ILP) can be decided via automata theory. In the absence of quantifier alternation (e.g., in ILP), this connection is often impractical: as shown in [Amat et al. 2025], automata-based tools are usually outperformed by algebraic methods on the QF_LIA benchmarks of the SMT-LIB repository [Barrett et al. 2016]. Automata can, however, be the best option on benchmarks with large coefficients and sparse solution sets [Ganesh et al. 2002]. For LIA formulas featuring quantifier

, Vol. 1, No. 1, Article . Publication date: July 2026.

integer. This is sometimes desirable in applications. A classic example is given by the decidability of the existential ($\exists^*$) fragment of LIA with divisibility constraints [Bel'tyukov 1976; Lipshitz 1978], which found applications in analysing programs with lists [Bozga and Iosif 2005], one-counter and timed automata [Bundala and Ouaknine 2017; Haase 2012; Pérez and Raha 2022], and in solving word equations [Lin and Majumdar 2021]. The algorithms from [Bel'tyukov 1976; Lipshitz 1978] require checking satisfiability of an $\exists^*$ formula Φ_p in the language of $(\mathbb{Q}_p; 0, 1, +, |_p, =)$ for every prime p not exceeding some constant. For a fixed prime p , the formula Φ_p can be translated in polynomial time into an $\exists^*$ formula of Büchi arithmetic [Guépin et al. 2019]. Motivated by this connection, [Haase and Mansutti 2021] shows that for $\exists^*$ sentences in Büchi arithmetic considered *parametrically in the base*, both the problems of satisfiability for *some* base $k \geq 2$ and for *all* bases $k \geq 2$ are decidable. The approach in [Haase and Mansutti 2021] does not handle quantifier alternation (it is known from [Haase and Różycki 2021; Starchak 2024] that existential Büchi arithmetic is less expressive than full Büchi arithmetic) but circumvents the fact that Theorem 1.1 is limited to a fixed base for the numeration system.

1.3 Our contributions

In this paper, we establish a parametric version of the Büchi-Bruyère theorem, which provides a unified framework for problems in which a given Büchi arithmetic formula must be solved either for a single large base, or for multiple bases. The correspondence between arithmetic theories and automatic structures that we establish is between *parametric Büchi arithmetic* and *parametric symbolic automata over LIA*. The former is a decidable syntactic fragment of the first-order theory of the structure $(\mathbb{Z}; 0, 1, +, V_t, \leq)$, consisting of Büchi arithmetic formulas where the base t is not a fixed integer constant but a free integer variable (the *parameter*). The latter are objects akin to the symbolic automaton in Figure 2b, but with the base k replaced by the parameter t . We leave the formal definitions to Section 2, but intuitively these objects represent functions $\mathcal{F}: \mathbb{N}_{\geq 2} \rightarrow \mathcal{P}(\mathbb{Z}^n)$ that map a base $k \geq 2$ to a set $\mathcal{F}(k) \subseteq \mathbb{Z}^n$; instead of a set of integer tuples, as in Theorem 1.1.

THEOREM 1.2 (PARAMETRIC RECOGNIZABILITY). *A function $\mathcal{F}: \mathbb{N}_{\geq 2} \rightarrow \mathcal{P}(\mathbb{Z}^n)$ is recognizable by a parametric symbolic automaton over LIA if and only if $\mathcal{F}$ is definable in parametric Büchi arithmetic.*

When a formula φ_t of parametric Büchi arithmetic does not have free variables except for the parameter t (i.e., it is a *sentence* of this logic, with $n = 0$ and $\mathbb{Z}^0 = \{\emptyset\}$), it represents a function $\mathcal{F}$ mapping $k \geq 2$ to $\{\emptyset\}$ whenever the Büchi arithmetic formula φ_k obtained by substituting t with k is a true sentence in the structure $(\mathbb{Z}; 0, 1, +, V_k, \leq)$. By Theorem 1.2, there is a parametric symbolic automaton $\mathcal{A}$ representing $\mathcal{F}$. Since the non-emptiness problem for the language of the automaton $\mathcal{A}$ is expressible by a LIA formula in the variable t , we obtain the following result.

COROLLARY 1.3. *For every sentence φ_t in parametric Büchi arithmetic, the set of bases $k \geq 2$ for which φ_k is a true sentence in the structure $(\mathbb{Z}; 0, 1, +, V_k, \leq)$ is an ultimately periodic set.*

This resolves a problem left open in [Haase and Mansutti 2021]: both “existence” and “universality” problems are decidable for the *full* Büchi arithmetic, not only the $\exists^*$ fragment.

In contrast to Theorem 1.1, where $\exists^*\forall^*$ formulas of Büchi arithmetic are sufficient to describe computations of finite automata [Haase and Różycki 2021; Villemaire 1992], the left-to-right direction of Theorem 1.2 requires $\exists^*\forall^*\exists^*$ formulas. We trace back this increase in quantifier alternation to the fact that, while the map $(x \mapsto k \cdot x)$ comes for free when the base is a fixed integer (it is expressible in LIA), $(x \mapsto t \cdot x)$ is undefinable in the parametric case. In Section 4 we analyse the effects of inclusion of multiplication by the parameter t into parametric Büchi arithmetic.

THEOREM 1.4. *The existence problem for the $\exists^*\forall^*$ fragment of parametric Büchi arithmetic extended with $(x \mapsto t \cdot x)$ is undecidable. Therefore, $(x \mapsto t \cdot x)$ is not definable in parametric Büchi arithmetic.*

This theorem is proved by encoding computations of counter machines via the bitwise minimum operation base t , similarly to [Jones and Matijasevič 1984; Starchak 2023].

In Section 5, we turn to the applicability of our theoretical results. First, we give an improved algorithm for translating linear inequalities into parametric symbolic automata. We then show how parametric Büchi arithmetic can express two arithmetical problems from the literature as instances of the BOUNDED EXISTENCE and BOUNDED UNIVERSALITY problems, i.e., where the base ranges over a finite interval $[\alpha.. \beta]$. We conclude the paper with an experimental comparison, on these bounded problems, of parametric versus non-parametric symbolic automata.

2 PRELIMINARIES

The naturals $\mathbb{N}$ include zero, and we write $\mathbb{N}_{\geq c}$ for the natural numbers greater than or equal to $c \in \mathbb{N}$. We usually use $a, b, c, \dots$ for integers. The powerset of a set S is denoted $\mathcal{P}(S)$. The set of all (total) functions from A to B is denoted $[A \rightarrow B]$. For a function $f: A \rightarrow B$ and a pair $(a, b) \in A \times B$, $f[a \mapsto b]: A \rightarrow B$ is the function identical to f on all inputs except that $f[a \mapsto b](a) = b$.

2.1 Logic notation

We assume familiarity with the basic definitions of first-order logic and only briefly recall some notions. A *signature* is a set of constant, function, and predicate symbols, together with their arities (which are usually clear from the context and omitted). Given a signature σ , we write $\mathcal{F}_\sigma$ for the set of all first-order formulas $\varphi, \psi, \dots$ over σ , using variables from a fixed countable set X . Formulas without free variables are called *sentences*. Given $m \geq 1$, the $\exists^* \forall^* \dots Q_\ell^*$ *fragment* of $\mathcal{F}_\sigma$ is understood as the set of all formulas of $\mathcal{F}_\sigma$ that, once rewritten in their prenex normal form, are of the form $Q_1 x_1 Q_2 x_2 \dots Q_\ell x_\ell \psi$ for some $\ell \leq m$, where $Q_i \in \{\exists, \forall\}$ stands for $\exists$ exactly when i is odd, each x_i is a vector of variables from X , and ψ is a quantifier-free formula. The $\exists^*$ fragment (of $\mathcal{F}_\sigma$) is sometimes called the *existential* fragment. Given real numbers $a, b \in \mathbb{R}$, we write $[a..b]$ for the set of integers $\{n \in \mathbb{Z} : a \leq n \leq b\}$, and $[a, b]$ for the real interval $\{x \in \mathbb{R} : a \leq x \leq b\}$.

A σ -*structure* $\mathfrak{M} := (M; I)$ is composed of a non-empty set M (i.e., the universe) and of an interpretation I mapping each constant/function/predicate symbol in σ to an element/function/relation over M . The *satisfaction relation* $\models_{\mathfrak{M}} _ : [X \rightarrow M] \rightarrow \mathcal{F}_\sigma \rightarrow \{\top, \perp\}$ is defined as usual for first order logic. For $n \in \mathbb{N}_{\geq 1}$, we say that a set $S \subseteq M^n$ is *definable in the structure* $\mathfrak{M}$ if there exist $\varphi \in \mathcal{F}_\sigma$ with n free variables $y_1, \dots, y_n \subseteq X$ such that $S = \{(\mu(y_1), \dots, \mu(y_n)) : \mu \models_{\mathfrak{M}} \varphi\}$. Since the satisfaction of a formula only depends on the values assigned to its free variables, for a formula φ with free variables among $x_1, \dots, x_n$, and $a_1, \dots, a_n \in M$, we will sometimes write $[x_1 \mapsto a_1, \dots, x_n \mapsto a_n] \models_{\mathfrak{M}} \varphi$ to denote that $\mu \models_{\mathfrak{M}} \varphi$ holds for all (equivalently, for some) valuation $\mu \in [X \rightarrow M]$ such that $\mu(x_i) = a_i$ for every $i \in [1..n]$. We will also sometimes use $\models_{\mathfrak{M}}$ as a unary relation on $\mathcal{F}_\sigma$, writing $\models_{\mathfrak{M}} \varphi$ whenever $\mu \models_{\mathfrak{M}} \varphi$ holds for every $\mu \in [X \rightarrow M]$. The *first-order theory of the structure* $\mathfrak{M}$ is the set $\text{Th}(\mathfrak{M}) := \{\varphi \in \mathcal{F}_\sigma : \varphi \text{ is a sentence and } \models_{\mathfrak{M}} \varphi\}$.

2.2 Parametric Büchi arithmetic

The formulas of *parametric Büchi arithmetic* are built from the signature $PBA := \{0, 1, +, V_t, \leq\}$, where $0, 1$, are constant symbols, $+$ is a binary function symbol, and V_t and $\leq$ are binary predicate symbols. We refer to the symbol $t \notin X$ as the *parameter*. Without loss of generality, we assume these first-order formulas to be built from the grammar

$$\varphi ::= \top \mid \perp \mid \sum_{i=1}^n a_i \cdot x_i \leq c \mid V_t(x, y) \mid \neg \varphi \mid \varphi \wedge \varphi \mid \exists x \varphi,$$

where $x, y, x_1, \dots, x_n$ are variables from X , and $a_1, \dots, a_n, c$ are integers encoded in binary. We use standard abbreviations for first-order logic, defining $\vee, \rightarrow, \leftrightarrow, \forall, <, \text{ and } =$ from $\neg, \wedge, \exists, \leq$.

Given $k \in \mathbb{N}_{\geq 2}$, consider the PBA-structure $\mathfrak{B}a_k := (\mathbb{Z}; I)$ interpreting the symbol V_t using the set $\{(n, d) \in \mathbb{Z} \times \mathbb{N} : d \text{ is the largest power of } k \text{ dividing } n\}$, and $0, 1, +$ and $\leq$ as the standard constants and operators over the integers. We call this *the structure of Büchi arithmetic on base k* , and write $\models_k$ as a shorthand for the satisfaction relation $\models_{\mathfrak{B}a_k}$. Observe that, for every $n, d \in \mathbb{Z}$, we have $V_t(n, d)$ if and only if $V_t(-n, d)$. The semantics $\llbracket _ \rrbracket_- : \mathcal{F}_{PBA} \rightarrow \mathbb{N}_{\geq 2} \rightarrow \mathcal{P}[X \rightarrow \mathbb{Z}]$ of parametric Büchi arithmetic is defined as the function that given a formula $\varphi \in \mathcal{F}_{PBA}$ and $k \in \mathbb{N}_{\geq 2}$ returns the set $\llbracket \varphi \rrbracket_k := \{\mu : \mu \models_k \varphi\}$ of the solutions that φ has in Büchi arithmetic on base k . We write $\mathcal{B}(\varphi)$ for the set of *realizable bases* of φ , i.e., the set $\{k \in \mathbb{N}_{\geq 2} : \llbracket \varphi \rrbracket_k \neq \emptyset\}$.

The EXISTENCE and UNIVERSALITY problems for parametric Büchi arithmetic ask, given a formula $\varphi \in \mathcal{F}_{PBA}$, whether $\mathcal{B}(\varphi)$ is non-empty or equal to $\mathbb{N}_{\geq 2}$, respectively.

REMARK 2.1. *The semantics $\llbracket _ \rrbracket_-$ given here is slightly different from the one considered in Section 1.3, which had the type $\mathcal{F}_{PBA} \rightarrow \mathbb{N}_{\geq 2} \rightarrow \mathcal{P}(\mathbb{Z}^n)$. Taking $\mathcal{P}[X \rightarrow \mathbb{Z}]$ instead of $\mathcal{P}(\mathbb{Z}^n)$ is entirely benign and allows us to avoid reordering variables in proofs and in various constructions. Formally, given a formula $\varphi(x_1, \dots, x_n) \in \mathcal{F}_{PBA}$ with free variables $x_1, \dots, x_n$, the semantics in Section 1.3 is obtained from $\llbracket \varphi \rrbracket_-$ as the map $(k \in \mathbb{N}_{\geq 2}) \mapsto \{(\mu(x_1), \dots, \mu(x_n)) \mid \mu \in \llbracket \varphi \rrbracket_k\}$.*

2.3 Presburger arithmetic with digits

We define *Presburger arithmetic with digits*¹ as the set of first-order formulas over the signature $PA_t := \{0, 1, +, t, \leq\}$. Given $k \in \mathbb{N}_{\geq 2}$, we denote by $\mathfrak{P}a_k := (\mathbb{Z}; J)$ the structure obtained from $\mathfrak{B}a_k$ by the replacement of the predicate V_t with the constant t interpreted by the integer k . The satisfaction relation $\models_k$ and semantics $\llbracket _ \rrbracket_-$ for Presburger arithmetic with digits are then defined analogously.

The qualifier “with digits” refers to the fact that, in this logic, the constant symbol t can be used to quantify over the digits $0, \dots, t-1$ of numbers encoded in base t : this is done with *bounded quantifiers* of the form $(\exists x < t) \varphi := \exists x(0 \leq x \leq t-1 \wedge \varphi)$. We define the *digit semantics* $\llbracket _ \rrbracket_-^{\#} : \mathcal{F}_{PA_t} \rightarrow (k : \mathbb{N}_{\geq 2}) \rightarrow \mathcal{P}[X \rightarrow [0..k-1]]$ of Presburger arithmetic with digits to be the function given by $\llbracket \varphi \rrbracket_k^{\#} := \llbracket \varphi \rrbracket_k \cap [X \rightarrow [0..k-1]]$, for every $\varphi \in \mathcal{F}_{PA_t}$ and $k \in \mathbb{N}_{\geq 2}$; i.e., $\llbracket \varphi \rrbracket_k^{\#}$ consists of all valuations μ mapping every variable to a digit base k , such that $k, \mu \models \varphi$.

2.4 Symbolic automata

For simplicity, we consider symbolic automata over first-order signatures, instead of Boolean algebras as in [D’Antoni and Veanes 2021]. Let σ be a signature.

Definition 2.1 (NSA). A *non-deterministic symbolic automaton* (over σ) is a tuple $(\Phi, Q, q_0, F, \delta)$ where: Φ is a finite set of first-order formulas from $\mathcal{F}_{\sigma}$ called *labels*, Q is a finite set of *states*, $q_0 \in Q$ is the *initial state*, $F \subseteq Q$ is the set of *accepting states*, and $\delta : Q \times Q \rightarrow \Phi$ is the *transition function*.

Consider a σ -structure $\mathfrak{M}$ with universe M . A *finite word* over $\mathfrak{M}$ is a sequence $\mathbf{w} = w_1 w_2 \dots w_n$ of valuations (called *letters*) of variables over $\mathfrak{M}$, that is, $\mathbf{w} \in [X \rightarrow M]^*$. We say that $\mathbf{w}$ *reaches state q from state p* whenever there is a sequence of states (i.e., a *path*) $p = q_1, \dots, q_{n+1} = q$ from Q such that $w_i \models_{\mathfrak{M}} \delta(q_i, q_{i+1})$ for every $i \in \{1, \dots, n\}$. A word $\mathbf{w}$ is *accepted by an NSA $\mathcal{A} := (\Phi, Q, q_0, F, \delta)$* whenever there $\mathbf{w}$ reaches $q_n \in F$ from the initial state q_0 . We write $\mathcal{L}_{\mathfrak{M}}(\mathcal{A})$ (or $\mathcal{L}(\mathcal{A})$ when $\mathfrak{M}$ is clear from the context) for the language of $\mathcal{A}$, i.e., the set of all finite words accepted by $\mathcal{A}$.

EXAMPLE 2.1. The reader has already seen a symbolic automaton over the formulas of LIA in Figure 2b. To make it formally defined, replace k with any fixed integer, and add the constraints $(0 \leq u \wedge u \leq k-1)$ for $u \in \{x, y, z\}$ to every outgoing transition of states 0 and 1. $\blacktriangleleft$

¹We use the name “Presburger arithmetic with digits” instead of “parametric Presburger arithmetic” to not confuse this logic with the one defined in [Bogart et al. 2019; Mansutti and Starchak 2025], which features a parametric multiplication function $x \mapsto t \cdot x$. We will discuss the effects of adding this function to parametric Büchi arithmetic in Section 4.

A symbolic automaton $\mathcal{A}$ is *deterministic* (for $\mathfrak{M}$)—in short, a DSA—whenever, for every $q \in Q$ and every $w: X \rightarrow M$, there is at most one $q' \in Q$ such that $w \models_{\mathfrak{M}} \delta(q, q')$. We recall standard closure properties of symbolic automata:

FACT 2.1 ([D'ANTONI AND VEANES 2021]). *Let σ be a signature, and let $\mathfrak{M}$ be a σ -structure. Consider symbolic automata over σ . There are algorithms with the following specifications:*

Determinization: *For a NSA $\mathcal{A}$, return a DSA $\mathcal{A}'$ such that $\mathcal{L}(\mathcal{A}') = \mathcal{L}(\mathcal{A})$.*

Intersection: *For NSAs $\mathcal{A}_1$ and $\mathcal{A}_2$, return an NSA $\mathcal{A}'$ such that $\mathcal{L}(\mathcal{A}') = \mathcal{L}(\mathcal{A}_1) \cap \mathcal{L}(\mathcal{A}_2)$.*

Complementation: *For a NSA $\mathcal{A}$, return an NSA $\mathcal{A}'$ such that $\mathcal{L}(\mathcal{A}') = [X \rightarrow M]^* \setminus \mathcal{L}(\mathcal{A})$.*

2.5 Number encodings

To encode integers as words that can be read by a (symbolic) automaton, we adopt the encoding of [Boigelot 2021]: a *most-significant-digit-first representation in k 's complement* (in short, *k-msdf*), for $k \in \mathbb{N}_{\geq 2}$. As explained in Section 1, in this representation words belong to the regular language $\mathcal{Z}_k := \{0, k-1\} \cdot \{0, 1, \dots, k-1\}^*$. The leftmost (most-significant) digit, constrained to 0 or $k-1$, is the *sign digit*.

For $\ell \geq 1$, the word $\mathbf{w} := a_\ell a_{\ell-1} \dots a_0 \in \mathcal{Z}_k$ represents the integer

$$\langle \mathbf{w} \rangle_k := \begin{cases} a_{\ell-1} \cdot k^{\ell-1} + \dots + a_1 \cdot k + a_0 & \text{if } a_\ell = 0, \\ -k^\ell + a_{\ell-1} \cdot k^{\ell-1} + \dots + a_1 \cdot k + a_0 & \text{if } a_\ell = k-1. \end{cases}$$

A key property of this representation is that padding words by their sign digits does not change the encoded number, since $-k^\ell = -k^{\ell+1} + (k-1) \cdot k^\ell$:

FACT 2.2. *For every $\mathbf{w} := a_\ell a_{\ell-1} \dots a_0 \in \mathcal{Z}_k$, we have $\langle a_\ell \mathbf{w} \rangle_k = \langle \mathbf{w} \rangle_k$.*

This makes it very simple to encode vectors of integers. Consider words $\mathbf{w}_1, \dots, \mathbf{w}_n$ from $\mathcal{Z}_k$, with $\mathbf{w}_i := a_{\ell_i}^{(i)} \dots a_0^{(i)}$, and let $m := \max(\ell_1, \dots, \ell_n)$, i.e., the maximal length of words in this list. Then, the *convolution* $\mathbf{w}_1 \otimes \dots \otimes \mathbf{w}_n := c_m \dots c_0$, where for every $i \in [0..m]$ the symbol $c_i \in \mathcal{Z}_k^n$ is defined as $c_i := (a_{\min(\ell_1, i)}^{(1)}, \dots, a_{\min(\ell_n, i)}^{(n)})$, encodes the n -tuple of integers $\langle \mathbf{w}_1 \otimes \dots \otimes \mathbf{w}_n \rangle_k := (\langle \mathbf{w}_1 \rangle_k, \dots, \langle \mathbf{w}_n \rangle_k)$.

A second property concerns the computation of the additive inverse $-n$ of a non-zero integer n . Directly from the definition of $\langle _ \rangle_k$, one can verify that:

FACT 2.3. *If $\mathbf{w} := a_\ell \dots a_0 \in \mathcal{Z}_k \setminus \{0\}^*$, then $-\langle \mathbf{w} \rangle_k = \langle (k-1-a_\ell)(k-1-a_{\ell-1}) \dots (k-1-a_0) \rangle_k + 1$.*

For example, in the decimal encoding we have $-\langle 0700 \rangle_{10} = -700 = -1000 + 299 + 1 = \langle 9299 \rangle_{10} + 1$.

We will soon need to consider words whose letters are maps from variables to $[0..k-1]$, rather than elements in $\mathcal{Z}_k$. Formally, we define the set $\mathcal{M}_k := [X \rightarrow \{0, k-1\}] \cdot [X \rightarrow [0..k-1]]^*$, i.e., the set of non-empty words $\mathbf{w} := w_\ell \dots w_0$ over the alphabet $[X \rightarrow [0..k-1]]$, such that $w_\ell \in [X \rightarrow \{0, k-1\}]$. As done for words in $\mathcal{Z}_k$, we will refer to w_ℓ as the *sign digit*. Note that, for every $x \in X$, the word $\mathbf{w}(x) := w_\ell(x) \dots w_0(x)$ belongs to $\mathcal{Z}_k$. Then, we can transform any word $\mathbf{w} \in \mathcal{M}_k$ into a map $\langle \mathbf{w} \rangle_k: X \rightarrow \mathbb{Z}$ by applying $\langle _ \rangle_k$ variable-wise, defining $\langle \mathbf{w} \rangle_k(x) := \langle w_\ell(x) \dots w_0(x) \rangle_k$ for every $x \in X$. In fact, $[X \rightarrow \mathbb{Z}] = \{\langle \mathbf{w} \rangle_k : \mathbf{w} \in \mathcal{M}_k\}$, for every $k \in \mathbb{N}_{\geq 2}$.

3 A PARAMETRIC BÜCHI-BRUYÈRE THEOREM

In this section, we establish Theorem 1.2. We first discuss the semantics of symbolic automata used in this theorem (Section 3.1), to then prove the right-to-left direction in Section 3.2. The left-to-right direction is proved in Section 3.3. Corollaries are given in Section 3.4. For simplicity of the exposition, the translation into symbolic automata provided in Section 3.2 is the simplest possible rather than the most efficient one. We leave the discussion of optimizations to Section 5, where we also discuss the implementation of the translation and an experimental evaluation.

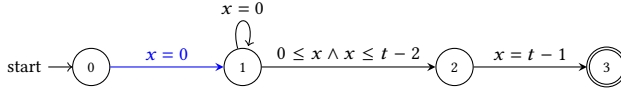
3.1 Parametric symbolic automata

Consider the class $\mathcal{Aut}(PA_t)$ of all non-deterministic symbolic automata with labels that are formulas from Presburger arithmetic with digits. We will now define a map

$$\mathcal{L}_{\text{msdf}} : \mathcal{Aut}(PA_t) \rightarrow (\mathbb{N}_{\geq 2} \rightarrow \mathcal{P}[X \rightarrow \mathbb{Z}])$$

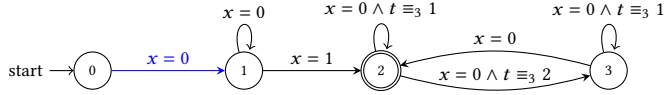
giving to these automata a semantic akin to that of parametric Büchi arithmetic. Roughly speaking, $\mathcal{L}_{\text{msdf}}$ will associate to $\mathcal{A} \in \mathcal{Aut}(PA_t)$ a function that, for a given base $k \in \mathbb{N}_{\geq 2}$, returns the set of assignments $X \rightarrow \mathbb{Z}$ whose k -msds encodings are accepted by the symbolic automaton obtained when replacing t by k . That is, through the semantic $\mathcal{L}_{\text{msdf}}$ a single automaton represents an infinite family of symbolic automata. When considering this semantic, we will call an automaton from $\mathcal{Aut}(PA_t)$ a *parametric symbolic automaton*.

EXAMPLE 3.1. Consider the parametric symbolic automaton $\mathcal{A}$:



where missing transitions are implicitly labeled with $\perp$. For example, substituting t by 3 in the transitions, we see that the words accepted by the resulting symbolic automaton are of the form $w_\ell \dots w_0 \in [X \rightarrow \mathbb{Z}]^*$ where $w_i(x) = 0$ for every $i \in [2..\ell]$, $w_1(x) \in [0..1]$ and $w_0(x) = 2$. Observe that $w_\ell(x) \dots w_0(x)$ represents an integer in 3-msdf; it is equal to 2 when $w_1(x) = 0$, or to 5 when $w_1(x) = 1$. Then, for every assignment $m \in \mathcal{L}_{\text{msdf}}(\mathcal{A})(3)$, we will have that $m(x) = 2$ or $m(x) = 5$. More generally, $\mathcal{L}_{\text{msdf}}(\mathcal{A})$ will be defined as the map that, given a $k \in \mathbb{N}_{\geq 2}$, returns the set of all assignments in $[X \rightarrow \mathbb{Z}]$ mapping x to an integer of the form $a \cdot k + (k-1)$, where $a \in [0..k-2]$. $\blacktriangleleft$

EXAMPLE 3.2. Consider the parametric symbolic automaton $\mathcal{B}$:



where for $r \in \{0, 1, 2\}$ the congruence $t \equiv_3 r$ is a shorthand for $\exists y (t = 3 \cdot y - r)$. This example illustrates a peculiarity of parametric symbolic automata compared to symbolic automata under the standard semantics: constraints on the parameter t propagate through a path. Taking the transition $2 \rightarrow 3$ requires $t \equiv_3 2$, which disables the self-loop on state 3. Similarly, the transition $3 \rightarrow 2$ is implicitly only taken when $t \equiv_3 2$, despite its label not mentioning t . The automaton $\mathcal{B}$ defines a map from $k \in \mathbb{N}_{\geq 2}$ to the set of all assignments in $[X \rightarrow \mathbb{Z}]$ that encodes, following Remark 2.1, the set $\{1\}$ if $k \equiv_3 0$, the set $\{k^n : n \in \mathbb{N}\}$ if $k \equiv_3 1$, and the set $\{k^{2n} : n \in \mathbb{N}\}$ if $k \equiv_3 2$. For instance, $m(x)$ is a power of 4 for every $m \in \mathcal{L}_{\text{msdf}}(\mathcal{B})(2)$, and it is a power of 10 for $m \in \mathcal{L}_{\text{msdf}}(\mathcal{B})(10)$. $\blacktriangleleft$

To formally define $\mathcal{L}_{\text{msdf}}$, start by considering an automaton $\mathcal{A} := (\Phi, Q, q_0, F, \delta)$ from $\mathcal{Aut}(PA_t)$, and some integer $k \geq 2$. By definition, the language $\mathcal{L}_{\mathfrak{p}_{a_k}}(\mathcal{A})$, where $\mathfrak{p}_{a_k}$ is the structure of Presburger arithmetic with digits from Section 2.3, is a set of words $w_1 \dots w_n$ where each letter w_i is a map from X to $\mathbb{Z}$. Let us define $\mathcal{L}_k(\mathcal{A}) := \mathcal{L}_{\mathfrak{p}_{a_k}}(\mathcal{A}) \cap \mathcal{M}_k$ with $\mathcal{M}_k$ from Section 2.5. Intuitively, these are the (non-empty) words that respect the constraints on the digits imposed by the msdf representation of integers in k 's complement. We call these the *words accepted by $\mathcal{A}$ in k -msdf*. We remark that the empty word is never in $\mathcal{L}_k(\mathcal{A})$, no matter whether q_0 belongs to F or not.

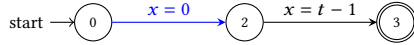
As explained in Section 2.5, every word accepted by $\mathcal{A}$ in k -msdf can be seen as a map from X to $\mathbb{Z}$, by applying the k -msdf representation of integers variable-wise: for $\mathbf{w} := w_\ell \dots w_0 \in \mathcal{L}_k(\mathcal{A})$, this is just the map $(\llbracket \mathbf{w} \rrbracket)_k : X \rightarrow \mathbb{Z}$. Finally, $\mathcal{L}_{\text{msdf}}(\mathcal{A})$ is defined as the map that for $k \in \mathbb{N}_{\geq 2}$ returns $\mathcal{L}_{\text{msdf}}(\mathcal{A})(k) := \{(\llbracket \mathbf{w} \rrbracket)_k : \mathbf{w} \in \mathcal{L}_k(\mathcal{A})\}$. We restate Theorem 1.2 using the introduced notation.

THEOREM (1.2). *The set of formulas of parametric Büchi arithmetic and the set of parametric symbolic automata represent the same class of functions from $[\mathbb{N}_{\geq 2} \rightarrow \mathcal{P}[X \rightarrow \mathbb{Z}]]$. That is, we have the equality $\{\mathcal{L}_{\text{msdf}}(\mathcal{A}) : \mathcal{A} \in \mathcal{Aut}(\text{Pat})\} = \{\llbracket \varphi \rrbracket : \varphi \in \mathcal{F}_{\text{PBA}}\}$.*

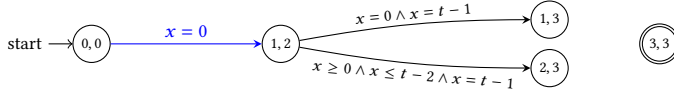
To prove this theorem, we are going to show that for every $\mathcal{A} \in \mathcal{Aut}(\text{Pat})$ there is $\varphi \in \mathcal{F}_{\text{PBA}}$ such that $\mathcal{L}_{\text{msdf}}(\mathcal{A}) = \llbracket \varphi \rrbracket$, and vice versa. For example, for the automaton $\mathcal{B}$ from Example 3.2 we have the equality $\mathcal{L}_{\text{msdf}}(\mathcal{B}) = \llbracket V_t(x, x) \wedge \exists y : x = 3 \cdot y + 1 \rrbracket$. Both directions of the proof are constructive: they yield algorithms to perform the translations.

3.2 From parametric Büchi arithmetic to symbolic automata

Before translating parametric Büchi arithmetic formulas into parametric symbolic automata, we must address an encoding issue. Observe that the automaton from Example 3.1 accepts, for any given $k \in \mathbb{N}_{\geq 2}$, the word $w_2 w_1 w_0$ with $w_2(x) = 0$, $w_1(x) = 0$ and $w_0(x) = k - 1$, while it rejects the word $u_1 u_0$ with $u_1(x) = 0$ and $u_0(x) = k - 1$. In both words, x encodes the same integer, since $\langle 00(k-1) \rangle_k = \langle 0(k-1) \rangle_k$ by Fact 2.2. This is problematic: consider the automaton $\mathcal{C}$ obtained from $\mathcal{A}$ by removing state (1) and adding a transition from state (0) to state (2) with label $x = 0$:



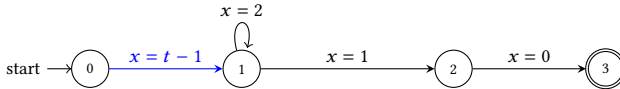
While the intersection of $\mathcal{L}_{\text{msdf}}(\mathcal{A})(k)$ and $\mathcal{L}_{\text{msdf}}(\mathcal{C})(k)$ contains the map sending x to $k - 1$, the algorithm for intersecting symbolic automata (from Fact 2.1; it just performs the Cartesian product construction) yields, when applied to $\mathcal{A}$ and $\mathcal{C}$, an automaton $\mathcal{D}$ with $\mathcal{L}_{\mathfrak{A}_k}(\mathcal{D}) = \emptyset$:



For the (standard) finite automata for Büchi arithmetic on a fixed base, this issue is resolved by requiring that if a word is accepted, then *all* its paddings by repeating the sign digits are also accepted [Boigelot 2021]. We adapt this notion to symbolic automata as follows:

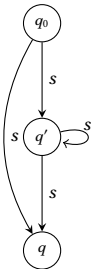
Definition 3.1 (Valid NSA). An NSA $\mathcal{A} \in \mathcal{Aut}(\text{Pat})$ is *valid* if it satisfies the following property for every $k \in \mathbb{N}_{\geq 2}$: for every map $\mu \in \mathcal{L}_{\text{msdf}}(\mathcal{A})(k)$ and word $\mathbf{w} \in \mathcal{M}_k$, if $\langle \mathbf{w} \rangle_k = \mu$, then $\mathbf{w} \in \mathcal{L}_k(\mathcal{A})$.

EXAMPLE 3.3. Consider the parametric symbolic automaton



This automaton does not satisfy the property in Definition 3.1 for $k = 2$ nor for $k \geq 4$, and it is thus not a valid NSA. Indeed, note that for $k = 2$ the word 110 is accepted, but not the words 1110 or 10, which encode the same integer. Similarly, for $k \geq 4$, the word $(k-1)210$ is accepted, but not the word $(k-1)(k-1)210$. However, the required property is satisfied for $k = 3$: in this case we have $\mathcal{L}_3(\mathcal{A}) = \{2^i 10 : i \in \mathbb{N}_{\geq 1}\}$, which is closed under padding by repeating the sign digit 2. $\triangleleft$

In [Boigelot 2021], finite automata for Büchi arithmetic are made valid by a simple procedure that “completes” the automaton by adding transitions to pad accepted words by repeating the sign digits. This procedure considers any state q reachable from the initial state q_0 by reading a word in the language s^+ , where s is a tuple of sign digits, and adds to the automaton the gadget on the right, where q' is a fresh state. In the completed automaton, q is guaranteed to be reachable from q_0 by any word in s^+ , instead of just some of them. A similar procedure also works for parametric symbolic automata, but we must take into account that, as shown in Example 3.3, the property in Definition 3.1 can be satisfied in some bases but not in others. The pseudocode is given in Algorithm 1.



Algorithm 1 Makes a parametric symbolic automaton valid.

Input: A NSA $\mathcal{A} = (\Phi, Q, q_0, F, \delta)$ from $\mathcal{Aut}(Pat)$

Output: A valid NSA $\mathcal{A}' \in \mathcal{Aut}(Pat)$ such that $\mathcal{L}_{msdf}(\mathcal{A}') = \mathcal{L}_{msdf}(\mathcal{A})$

- 1: **let** $\{x_1, \dots, x_m\} \subseteq X$ be the set of free variables occurring in formulas from Φ
 - 2: **let** $Q' := \{q'_0\}$ where q'_0 is a fresh state
 - 3: **for** every map $\sigma: \{x_1, \dots, x_m\} \rightarrow \{0, t-1\}$ **do**
 - 4: **for** every path $q_0 q_1 \dots q_n \in Q^*$ such that $q_1, \dots, q_n$ are pairwise distinct **do**
 - 5: **let** q'' be a fresh state, and ψ be the formula $(\bigwedge_{i=1}^m x_i = \sigma(x_i)) \wedge \bigwedge_{i=0}^{n-1} \delta(q_i, q_{i+1})$
 - 6: Add q'' to Q' , and add ψ to Φ
 - 7: Extend δ by adding entries $(q'_0, q_n) \mapsto \psi$, $(q'_0, q'') \mapsto \psi$, $(q'', q'') \mapsto \psi$ and $(q'', q_n) \mapsto \psi$
 - 8: Complete δ on the domain $(Q \cup Q') \times (Q \cup Q')$ by mapping missing pairs to $\perp$
 - 9: **return** $(\Phi, Q \cup Q', q'_0, F, \delta)$
-

LEMMA 3.2. *Algorithm 1 respects its specification.*

PROOF IDEA. Let us informally discuss Algorithm 1, leaving the formal proof of Lemma 3.2 to Appendix A. Lines 3 and 4 of the algorithm enumerate all possible maps of sign digits, and all possible paths of the automaton (disregarding loops, except those involving the initial state q_0). Consider, for example, the map $x \mapsto t-1$ and the path $0 \rightarrow 1 \rightarrow 2$ in the automaton from Example 3.3. Suppose we were to add a gadget similar to that of [Boigelot 2021], just with the labels “s” replaced by $x = t-1$. This would fix the automaton in Example 3.3 for $k = 2$ (now $\mathcal{L}_2(\mathcal{A}) = \{1^i 0 : i \in \mathbb{N}_{\geq 1}\}$), but inadvertently changes $\mathcal{L}_3(\mathcal{A})$ to also include words from 2^+0 . Instead, Algorithm 1 gathers all formulas along the considered path, and strengthens the labels of the gadget with their conjunction. In our example, the path $0 \rightarrow 1 \rightarrow 2$ carries the labels $x = t-1$ (redundant) and $x = 1$, so the gadget’s transitions are instead labeled with $x = t-1 \wedge x = 1$. This formula is only satisfiable for $t = 2$, and thus the gadget does not change the language of the automaton in other bases. $\square$

We can thus always assume that parametric symbolic automata are valid. On these automata, the basic algorithms for symbolic automata work as expected with respect to the semantics of $\mathcal{L}_{msdf}$

LEMMA 3.3. *There are algorithms satisfying the following specifications for automata in $\mathcal{Aut}(Pat)$:*

Determinization: *For a valid NSA $\mathcal{A}$, returns a valid DSA $\mathcal{A}'$ such that $\mathcal{L}_{msdf}(\mathcal{A}') = \mathcal{L}_{msdf}(\mathcal{A})$.*

Intersection: *For valid NSAs $\mathcal{A}_1$ and $\mathcal{A}_2$, return a valid NSA $\mathcal{A}'$ such that for every $k \in \mathbb{N}_{\geq 2}$, $\mathcal{L}_{msdf}(\mathcal{A}')(k) = \mathcal{L}_{msdf}(\mathcal{A}_1)(k) \cap \mathcal{L}_{msdf}(\mathcal{A}_2)(k)$.*

Complementation: *For a valid NSA $\mathcal{A}$, returns a valid NSA $\mathcal{A}'$ such that for every $k \in \mathbb{N}_{\geq 2}$, $\mathcal{L}_{msdf}(\mathcal{A}')(k) = M_k \setminus \mathcal{L}_{msdf}(\mathcal{A})(k)$.*

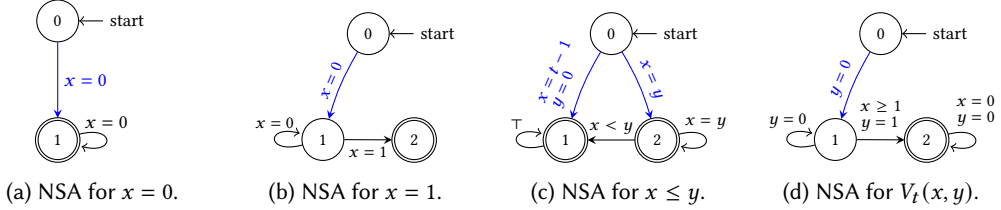
To translate parametric Büchi arithmetic formulas into parametric symbolic automata, it now suffices to show closure under variable projection, and to provide translations for the atomic formulas. Let us start with variable projection. Given a variable $x \in X$, we define $\exists_x: \mathcal{Aut}(Pat) \rightarrow \mathcal{Aut}(Pat)$ as the map guarding every formula in the input NSA with the bounded quantifier $(\exists x < t)$. Formally, given NSA $\mathcal{A} = (\Phi, Q, q_0, F, \delta)$, we have $\exists_x(\mathcal{A}) := (\Phi', Q, q_0, F, \delta')$ where $\Phi' := \{(\exists x < t) \varphi : \varphi \in \Phi\}$ and $\delta'(q, q') := (\exists x < t) \delta(q, q')$ for every $q, q' \in Q$. This map ensures closure under projection:

LEMMA 3.4. *Consider $\mathcal{A} \in \mathcal{Aut}(Pat)$ and $x \in X$. Then, for every $k \in \mathbb{N}_{\geq 2}$ and $m \in [X \rightarrow \mathbb{Z}]$, $m \in \mathcal{L}_{msdf}(\exists_x(\mathcal{A}), k)$ if and only if there is $j \in \mathbb{Z}$ such that $m[x \mapsto j] \in \mathcal{L}_{msdf}(\mathcal{A})(k)$.*

Note that the output of $\exists_x$ is not guaranteed to be a valid parametric symbolic automaton.

Thanks to closure under projection, translating arbitrary atomic formulas reduces to handling the following base cases: $x = 0$, $x = 1$, $x \leq y$, $V_t(x, y)$ and $x + y = z$ (arbitrary atomic formulas can

be expressed with existential formulas over these base cases). We have already encountered the parametric symbolic automaton for $x + y = z$: it is the one obtained from Figure 2b by replacing all occurrences of k by the parameter t . Here are the remaining base cases:



It is routine to verify that all the NSAs for the base cases are correct. The right-to-left direction of Theorem 1.2 then follows by structural induction on $\varphi \in \mathcal{F}_{PBA}$, using Lemmas 3.2 to 3.4.

LEMMA 3.5. *For every formula φ in $\mathcal{F}_{PBA}$ there is a NSA $\mathcal{A} \in \mathcal{Aut}(PA_t)$ with $\mathcal{L}_{msdf}(\mathcal{A}) = \llbracket \varphi \rrbracket$.*

3.3 Arithmetization of parametric symbolic automata

We now prove the other direction of Theorem 1.2: computations of parametric symbolic automata can be represented by formulas of parametric Büchi arithmetic. In Büchi-Bruyère theorem [Bruyère et al. 1994; Villemaine 1992], the analogous statement is obtained via a translation yielding a formula $\varphi \in \mathcal{F}_{kBA}$ with a bounded number of quantifier alternations. A slight modification shows that φ can be taken from the $\exists^*\forall^*$ fragment of $\mathcal{F}_{kBA}$ [Haase and Różycki 2021].

We need a few auxiliary predicates. The first two are simple: $P_t(x) := V_t(x, x)$ states that x is a (positive) power of t , and $S_t(x, y) := P_t(x) \wedge P_t(y) \wedge x < y \wedge \forall z (P_t(z) \rightarrow z \leq x \vee y \leq z)$ states that x and y are successive powers of t , i.e., $x = t^i$ and $y = t^{i+1}$ for some $i \in \mathbb{N}$. We will use S_t to compare digits in the base t expansions of integers. Unlike Büchi arithmetic on base k , where $S_k(x, y)$ can simply be defined as $P_k(x) \wedge y = k \cdot x$, here S_t uses universal quantifiers. We will see in Section 4 that the multiplication “ $y = t \cdot x$ ” is not definable in $\mathcal{F}_{PBA}$. This will ultimately limit our translation to the $\exists^*\forall^*\exists^*$ fragment instead of $\exists^*\forall^*$.

Next, we define a predicate $\text{digit}_t(x, p, p', y)$ with the following semantics: for every $k \in \mathbb{N}_{\geq 2}$ and $\mathbf{w} := w_\ell \dots w_0 \in \mathcal{M}_k$ such that $\langle \mathbf{w} \rangle_k(p) = k^i$ and $\langle \mathbf{w} \rangle_k(p') = k^{i+1}$ for some $i \in \mathbb{N}$, we have $\langle \mathbf{w} \rangle_k \models \text{digit}_t(x, p, p', y)$ if and only if $\langle \mathbf{w} \rangle_k(y) = d \cdot k^i$ and $w_{\min(\ell, i)}(x) = d$ for some $d \in [0..k-1]$. In other words, $\text{digit}_t(x, p, p', y)$ is true whenever $y = d \cdot t^i$ (shorthand for “the integer assigned to y is $d \cdot t^i$ ”) and d is the i -th digit of x (assuming $p = t^i$ and $p' = t^{i+1}$). This is, in some sense, a “parametric generalization” of the predicate $\in_{d,k}(x, y)$ from [Bruyère et al. 1994].

LEMMA 3.6. *The predicate $\text{digit}_t(x, p, p', y)$ is definable in both $\exists^*$ and $\forall^*$ fragments of $\mathcal{F}_{PBA}$.*

PROOF. For simplicity, we start by considering non-negative integers only. Assuming $p = t^i$ and $p' = t^{i+1}$, the following quantifier-free formula $\text{split}_t(x, p, p', p'', x_1, x_2, x_3)$ decomposes x into three parts x_1 , x_2 and x_3 , where x_1 contains digits less significant than i , x_3 contains digits more significant than i , and x_2 is exactly the “monomial” $d \cdot t^i$ we are after. The auxiliary variable p'' is a large power of t used to capture the digits more significant than i .

$$\text{split}_t(x, p, p', p'', x_1, x_2, x_3) := (x = x_1 + x_2 + x_3) \wedge \bigwedge_{j \in \{1, 2, 3\}} (0 \leq x_j) \wedge (x_1 < p) \wedge \\ (x_2 = 0 \vee (V_t(x_2, p) \wedge x_2 < p')) \wedge (x_3 = 0 \vee (p' \leq p'' \wedge V_t(x_3, p''))).$$

There is a unique decomposition of x into the sum of x_1 , x_2 and x_3 . Defining digit_t on the non-negative integers, denoted digit_t^+ , can be done in both the $\exists^*$ and $\forall^*$ fragment of $\mathcal{L}_{PBA}$ as follows:

$$\text{digit}_t^+(x, p, p', y) := \exists p'', x_1, x_3 \text{ split}_t(x, p, p', p'', x_1, y, x_3), \\ \text{or } \text{digit}_t^+(x, p, p', y) := \forall p'', x_1, x_2, x_3 (\text{split}_t(x, p, p', p'', x_1, x_2, x_3) \rightarrow x_2 = y).$$

One then relies on Fact 2.3 to extend the definition to the negative numbers:

$$\text{digit}_t(x, p, p', y) := ((x < 0) \wedge \text{digit}_t^+(-x - 1, p, p', p' - p - y)) \vee ((0 \leq x) \wedge \text{digit}_t^+(x, p, p', y)). \quad \square$$

The last predicate we need before describing the translation, denoted local_φ , merits an informal explanation. The formulas form $\mathcal{F}_{\text{Pat}}$ labeling transitions of the automata read only one digit at a time. However, parametric Büchi arithmetic has no way to extract a digit directly: we can only extract monomials $d \cdot t^i$, as done by the formula digit_t . The formula local_φ resolves this mismatch by “shifting” the solutions of φ by an arbitrary power of t . Its formal semantics is given below.

LEMMA 3.7. *Let $\varphi(x_1, \dots, x_n)$ be from $\mathcal{F}_{\text{Pat}}$. We can construct a formula $\text{local}_\varphi(p, p', x_1, \dots, x_n)$, in the $\exists^*$ fragment of $\mathcal{F}_{\text{PBA}}$ with the property: for every $k \in \mathbb{N}_{\geq 2}$, and $\mathbf{w} := w_\ell \dots w_0 \in \mathcal{M}_k$, if $\langle \mathbf{w} \rangle_k(p) = k^i$ and $\langle \mathbf{w} \rangle_k(p') = k^{i+1}$ for some $i \in \mathbb{N}$, then $w_i \models_k \varphi \iff \langle \mathbf{w} \rangle_k \models_k \text{local}_\varphi$.*

PROOF. To obtain an existential formula, we rely on the fact that Presburger arithmetic has quantifier elimination in the extended structure $(\mathbb{Z}, 0, 1, +, (\equiv_d)_{d \in \mathbb{N}_{\geq 2}}, \leq)$, where $\equiv_d$ stands for the congruence modulo d relation [Presburger 1929]:

FACT 3.1. *The structure $(\mathbb{Z}, 0, 1, +, (\equiv_d)_{d \in \mathbb{N}_{\geq 2}}, \leq)$ admits quantifier elimination: there is an algorithm that, given a first-order formula φ over this structure, returns an equivalent quantifier-free formula ψ .*

Seeing the parameter t as a standard integer variable, we compute a quantifier-free formula ψ equivalent to φ . Without loss of generality, we assume ψ to be a *positive* (i.e., negation-free) Boolean combination of constraints $\tau \sim 0$, where τ is a linear term of the form $a_1 \cdot x_1 + \dots + a_n \cdot x_n + b \cdot t + c$, with a_i, b and c integers, and $\sim$ is either $\leq$ or $\equiv_d$ for some $d \in \mathbb{N}_{\geq 2}$. Positivity can be forced thanks to the equivalences $\neg(\tau \leq 0) \leftrightarrow (-\tau + 1 \leq 0)$ and $\neg(\tau \equiv_d 0) \leftrightarrow \bigvee_{r \in [1, d-1]} \tau - r \equiv_d 0$.

We now “shift” the solutions to ψ , constructing a formula $\psi^*(y_1, \dots, y_n)$ such that, when each y_j is set to a monomial $d_j \cdot t^i$, ψ^* is satisfied if and only if so is $\varphi(d_1, \dots, d_n)$. Recall that $\langle \mathbf{w} \rangle_k(p) = k^i$ and $\langle \mathbf{w} \rangle_k(p') = k^{i+1}$. The formula ψ^* is obtained by manipulating ψ as explained below.

Inequalities $a_1 \cdot x_1 + \dots + a_n \cdot x_n + b \cdot t + c \leq 0$ are rewritten to $a_1 \cdot y_1 + \dots + a_n \cdot y_n + b \cdot p' + c \cdot p \leq 0$. Intuitively, every monomial in the original inequality is multiplied by p , after which each term $x_j \cdot p$ is replaced by y_j , and $t \cdot p$ is replaced by p' .

For a congruence $\tau \equiv_d 0$, we can similarly start by multiplying everything by p , obtaining $\tau \cdot p \equiv_{d \cdot p} 0$. This is not, however, a formula in $\mathcal{F}_{\text{PBA}}$, since the modulus is no longer a constant. Let us nonetheless proceed as for inequalities, replacing $x_j \cdot p$ by y_j , and $t \cdot p$ by p' . We obtain a congruence of the form $a_1 \cdot y_1 + \dots + a_n \cdot y_n + b \cdot p' + c \cdot p \equiv_{d \cdot p} 0$. This congruence is equivalent to $\exists y (d \cdot p \cdot y = a_1 \cdot y_1 + \dots + a_n \cdot y_n + b \cdot p' + c \cdot p)$, where y ranges over $\mathbb{Z}$. But $p \cdot y$ is nothing more than an integer that is divisible by p , and so this formula is equivalent to the $\mathcal{F}_{\text{PBA}}$ formula

$$\exists y \exists p'' (d \cdot y = a_1 \cdot y_1 + \dots + a_n \cdot y_n + b \cdot p' + c) \wedge V_t(y, p'') \wedge p \leq p''.$$

Once the formula ψ^* has been constructed following the transformations above, we define:

$$\text{local}_\varphi(p, p', x_1, \dots, x_n) := \exists y_1 \dots \exists y_n \psi^*(p, p', y_1, \dots, y_n) \wedge \bigwedge_{i=1}^n \text{digit}_t(x_i, p, p', y_i).$$

The fact that local_φ is correct and is an existential formula then follows from Lemma 3.6. $\square$

With these preparation steps done, we can proceed to translate automata.

LEMMA 3.8. *For every $\mathcal{A} \in \mathcal{Aut}(\text{Pat})$, we can compute a formula φ in the $\exists^* \vee^*$ fragment of parametric Büchi arithmetic extended with the predicate S_t , such that $\mathcal{L}_{\text{msdf}}(\mathcal{A}) = \llbracket \varphi \rrbracket$.*

PROOF. Let $\mathcal{A} = (\Phi, Q, q_0, F, \delta)$ be a parametric symbolic automaton from $\mathcal{Aut}(\text{Pat})$, and let $\mathbf{x} = (x_1, \dots, x_n)$ be the variables appearing in the formulas of Φ . For every state $q \in Q$, we introduce an integer variable with the same name, and denote these variables collectively as $\mathbf{q}$. The intended

reading is that, for a fixed base t , the variables $\mathbf{q}$ jointly describe a single run of $\mathcal{A}$: at every position i , exactly one variable $q \in Q$ has digit 1 in its base- t expansion at position i , other variables from Q have digit 0, and this q is the state that the run occupies just after consuming the digit at position i from the input. To make this more precise, we need a way to identify the length of the run. Fix a base t and consider a run $q_0 \xrightarrow{w_m} q_1 \xrightarrow{w_{m-1}} \dots \xrightarrow{w_0} q_{m+1}$ of $\mathcal{A}$, on a word $\mathbf{w} = w_m \dots w_0 \in \mathcal{M}_k$ accepted by $\mathcal{A}$; here q_j , for $j \in [0..m+1]$, is the state reached after reading the j symbols $w_m, \dots, w_{m-j+1}$, so that q_j is exactly the state we want active at digit position $m+1-j$. We want to encode this run using the variables $\mathbf{q}$, for an arbitrary base t and an arbitrary (unknown) length $m+1$. Since m cannot be referred to directly in our formula, we instead introduce an auxiliary variable u standing for t^m , together with two further variables u' and u'' standing for t^{m+1} and t^{m+2} , respectively.

For $q \in Q$, denote by $q[0], q[1], \dots$ the sequence of digits in $[0..t-1]$ such that $q = \sum_{i \in \mathbb{N}} q[i] \cdot t^i$. Formalizing the intuition above, we want the variables $\mathbf{q}$ to satisfy:

- (1) for every $i \in [0..m+1]$ exactly one $q \in Q$ is such that $q[i] = 1$, the others have a zero digit;
- (2) $q_0[m+1] = 1$, and $q[0] = 1$ for some $q \in F$.

Item 1 forces $\mathbf{q}$ to describe, at every position from 0 to $m+1$, a single well-defined “currently active” state. Item 2 fixes the two endpoints of the sequence. Assuming $u' = t^{m+1}$ and $u'' = t^{m+2}$, we capture Item 1 with the formula $\varphi_1(\mathbf{q}, u', u'')$ defined as

$$\bigwedge_{q \in Q} (0 \leq q < u'') \wedge \forall s \forall s' \left(S_t(s, s') \wedge s \leq u' \rightarrow \bigvee_{q \in Q} (\text{digit}_t(q, s, s', s) \wedge \bigwedge_{q' \in Q \setminus \{q\}} \text{digit}_t(q', s, s', 0)) \right).$$

The first part states that the variables $q \in Q$ may have only non-negative values and that their most significant bit is at most $(m+1)$. The second part uses the predicate digit_t to assert that at every position $i \in [0..m+1]$ exactly one state is active (with digit one), and the remaining ones are inactive (with digit zero). We capture Item 2 with the formula $\varphi_2(\mathbf{q}, u', u'')$ defined as

$$\text{digit}_t(q_0, u', u'', u') \wedge \forall v (S_t(1, v) \rightarrow \bigvee_{q \in F} \text{digit}_t(q, 1, v, 1)).$$

Note that, in this formula, the premise of the implication can be true only when the variable v is equal to t . Both formulas $\varphi_1(\mathbf{q}, u', u'')$ and $\varphi_2(\mathbf{q}, u', u'')$ only feature universal quantifiers (when using the $\forall^*$ definition of digit_t from Lemma 3.6). Below, let $\text{run}_{Q,F}(\mathbf{q}, u', u'') := (\varphi_1 \wedge \varphi_2)$.

The variables $\mathbf{x} = (x_1, \dots, x_n)$ appearing in formulas from Φ are also the free variables of our final formula. So far, $\text{run}_{Q,F}(\mathbf{q}, u', u'')$ only constrains $\mathbf{q}$ to encode *some* sequence of states, without tying this sequence to the transition relation δ of $\mathcal{A}$ or to the values of $\mathbf{x}$. To capture a valid run and not just an arbitrary sequence of states, we conjoin to $\text{run}_{Q,F}(\mathbf{q}, u', u'')$ a further formula, denoted $\Delta(\mathbf{q}, u', \mathbf{x})$, which asserts that consecutive active states are indeed connected by a transition of δ labelled consistently with the digits of $\mathbf{x}$. This formula is defined as follows,

$$\Delta(\mathbf{q}, u', \mathbf{x}) := \forall s \forall s' \forall s'' (S_t(s, s') \wedge S_t(s', s'') \wedge s \leq u') \rightarrow \bigvee_{q, q' \in Q} (\text{local}_{\delta(q, q')}(s, s', \mathbf{x}) \wedge \text{digit}_t(q, s', s'', s') \rightarrow \text{digit}_t(q', s, s', s)).$$

Here s, s', s'' range, via $S_t(s, s') \wedge S_t(s', s'')$, over triples of consecutive powers of t , i.e. $s = t^i$, $s' = t^{i+1}$, $s'' = t^{i+2}$ for some position $i \in [0..m]$; the bound $s \leq u'$ restricts i to the range covered by the run. Recall that the run visits the state active at position $i+1$ *before* the state active at position i , since positions are read from most to least significant. Accordingly, Δ asserts that whenever a state q is active at the more significant position $i+1$ ($\text{digit}_t(q, s', s'', s')$) and the transition label $\delta(q, q')$ is satisfied locally at position i by the digits of $\mathbf{x}$ ($\text{local}_{\delta(q, q')}(s, s', \mathbf{x})$), the successor state q' must indeed be the state active at position i ($\text{digit}_t(q', s, s', s)$).

The formula $\Delta(q, u', x)$ lies in the $\forall^*$ fragment of parametric Büchi arithmetic with S_t : note that $\text{local}_{\delta(q, q')}$ and the first digit_t conjunct occur negated (as antecedents of the inner implication), so their $\exists^*$ definitions in Lemmas 3.6 and 3.7 turn into $\forall^*$ definitions. The remaining, positive occurrence of digit_t is instantiated with its $\forall^*$ definition from Lemma 3.6.

We can now give the final $\exists^*\forall^*$ formula $\varphi(x)$:

$$\exists q, u, u', u'' S_t(u, u') \wedge S_t(u', u'') \wedge \bigwedge_{x \in x} (u > \|x\|) \wedge \text{run}_{Q, F}(q, u', u'') \wedge \Delta(q, u', x),$$

where $(u > \|x\|) := (x > 0 \wedge u > x) \vee (x = 0 \wedge u > 1) \vee (x < 0 \wedge u > -x - 1)$ ensures that the most significant non-sign digit of x occurs on position at most $m - 1$, with $u = t^m$. This guarantees that the most significant digit of x that the formula sees is, in msdf, either a 0 (and x is non-negative) or a $t - 1$ (and x is negative), which is exactly what the semantics of $\mathcal{L}_{\text{msdf}}(\mathcal{A})$ requires. $\square$

Recalling that S_t can be defined in the $\forall^*$ fragment of parametric Büchi arithmetic, we obtain the following corollary, which completes the proof of Theorem 1.2.

COROLLARY 3.9. *For every $\mathcal{A} \in \mathcal{Aut}(Pat)$, we can compute a formula φ in the $\exists^*\forall^*\exists^*$ fragment of parametric Büchi arithmetic such that $\mathcal{L}_{\text{msdf}}(\mathcal{A}) = \llbracket \varphi \rrbracket$.*

3.4 Decision problems on parametric Büchi arithmetic

From Theorem 1.2 we can conclude that the set of bases for which a parametric Büchi arithmetic formula is satisfiable is ultimately periodic. A subset $S \subseteq \mathbb{N}$ is *ultimately periodic* if there are integers N and $p \geq 1$ such that for every $n \geq N$, $n \in S$ if and only if $n + p \in S$. Observe that such a set can be represented finitely by a triple (B, N, p) with N and p as in the definition, and $B \subseteq [0..N + p - 1]$ satisfying $B = S \cap [0..N + p - 1]$. Ultimately periodic sets are closed under union, intersection, and complementation, and these operations can be performed effectively on their representations (see, e.g., [Haase and Mansutti 2021, Proposition 9]).

We now describe an algorithm that, given in input a parametric Büchi arithmetic formula φ , computes an ultimately periodic representation of the set $\mathcal{B}(\varphi)$ of realizable bases of φ . The algorithm is based on Fact 3.1 and the following classic fact from [Ginsburg and Spanier 1966].

FACT 3.2. *The set of solutions of a univariate quantifier-free formula over $(\mathbb{Z}, 0, 1, +, (\equiv_d)_{d \in \mathbb{N}_{\geq 2}}, \leq)$ is ultimately periodic, and a representation of this set can be computed effectively from the formula.*

The pseudocode of the procedure is given in Algorithm 2. The algorithm starts by translating the input formula φ into a parametric symbolic automaton $\mathcal{A}$, following Lemma 3.5. Since in the k -msdf encoding the first digit is a sign digit in $[X \rightarrow \{0, k - 1\}]$, the algorithm changes the initial state to a fresh state $\tilde{q}_0$, and modifies the transition relation δ to make sure that the automaton only reads words whose first letter is a sign digit; see the constraints $x_i = 0 \vee x_i = t - 1$. The updates to δ also ensure that the initial state is not a final state, since in the k -msdf encoding the empty word is never accepted. Next (lines 4–7), the algorithm computes for each pair of states $q, q' \in Q$ an ultimately periodic set $b(q, q')$ characterizing the bases $k \geq 2$ such that $\delta(q, q')$ is satisfiable over $\mathfrak{Pa}_k$. This set is computed by relying on Facts 3.1 and 3.2, seeing t as a free integer variable instead of the constant symbol from the language of Presburger arithmetic with digits. The last part of the algorithm (lines 4–7) performs a fixpoint computation to populate a map m . Once stabilized, this map associates to each state $q \in Q$ an ultimately periodic set $m(q)$ characterizing the bases $k \geq 2$ for which there is a run from q to some final state in $\mathcal{A}$ over a word with letters in $[X \rightarrow [0..k - 1]]$; moreover, for $\tilde{q}_0$, these words must start with a sign digit. The map is initialized so that $m(q)$ represents $\mathbb{N}_{\geq 2}$ if q is a final state, and $\emptyset$ otherwise. Line 11 updates $m(q)$ by adding to it all bases $k \in b(q, q') \cap m(q')$, for all $q' \in Q$. Upon reaching a fixpoint, $m(\tilde{q}_0) = \mathcal{B}(\varphi)$.

LEMMA 3.10. *Algorithm 2 respects its specification and terminates.*

Algorithm 2 Computes the realizable bases of a parametric Büchi arithmetic formula.

Input: A formula $\varphi(\mathbf{x})$ in $\mathcal{F}_{PBA}$, with $\mathbf{x} = (x_1, \dots, x_m)$

Output: A representation of $\mathcal{B}(\varphi)$ as an ultimately periodic set

- 1: **let** $\mathcal{A} := (\Phi, Q, q_0, F, \delta)$ be an NSA in $\mathcal{Aut}(PA\mathcal{t})$ such that $\mathcal{L}_{msdf}(\mathcal{A}) = \llbracket \varphi \rrbracket$.
 - 2: add a fresh state $\tilde{q}_0$ to Q , and make it the initial state of $\mathcal{A}$
 - 3: extend δ , adding $(\tilde{q}_0, q) \mapsto \delta(q_0, q) \wedge \bigwedge_{i=1}^m (x_i = 0 \vee x_i = t - 1)$ and $(q, \tilde{q}_0) \mapsto \perp$ for all $q \in Q$
 - 4: **let** b be an empty map from $Q \times Q$ to ultimately periodic sets
 - 5: **for** every pair of states $q, q' \in Q$ **do**
 - 6: **let** $\psi(t)$ be the quantifier-free formula of Presburger arithmetic obtained by performing quantifier elimination on $t \geq 2 \wedge (\exists \mathbf{x} < t) \delta(q, q')$, seeing t as a (free) integer variable
 - 7: $b(q, q') \leftarrow$ ultimately periodic set representation of the solutions of $\psi(t)$
 - 8: **let** m be the map from Q to ultimately periodic sets given by $m(q) := \begin{cases} (\{2\}, 2, 1) & \text{if } q \in F \\ (\emptyset, 0, 1) & \text{if } q \in Q \setminus F \end{cases}$
 - 9: **while true do**
 - 10: **let** m' be a copy of m
 - 11: **for** every state $q \in Q$ **do** $m(q) \leftarrow m(q) \cup \bigcup_{q' \in Q} (b(q, q') \cap m(q'))$
 - 12: **if** $m = m'$ **then return** $m(\tilde{q}_0)$
-

PROOF SKETCH. The correctness of the algorithm follows from the discussion above; we refer to Appendix B for a formal proof. For termination, consider the set U of ultimately periodic sets generated by taking unions and intersections of the sets $b(q, q')$, $(\{2\}, 2, 1)$, and $(\emptyset, 0, 1)$, for all $q, q' \in Q$. The set U is a partially ordered set with respect to the subset relation $\subseteq$. Also define $\dot{\subseteq}$ as the point-wise order on $[Q \rightarrow U]$ given by $m \dot{\subseteq} m'$ if and only if $m(q) \subseteq m'(q)$ for every $q \in Q$. Note that $[Q \rightarrow U]$ is a partially ordered set with respect to $\dot{\subseteq}$.

The body of the while loop corresponds to an operator $\Phi: [Q \rightarrow U] \rightarrow [Q \rightarrow U]$ that updates the current map m . It is defined as $\Phi(m)(q) := m(q) \cup \bigcup_{q' \in Q} (b(q, q') \cap m(q'))$. This operator is inflationary, i.e., $m \dot{\subseteq} \Phi(m)$ for every $m \in [Q \rightarrow U]$. Hence, regardless of how m is initialized, iterates of Φ form an ascending chain in $[Q \rightarrow U]$. Since $[Q \rightarrow U]$ is finite, this chain must reach a fixpoint after finitely many iterations, and the while loop terminates. $\square$

COROLLARY 1.3. *For every sentence φ_t in parametric Büchi arithmetic, the set of bases $k \geq 2$ for which φ_k is a true sentence in the structure $(\mathbb{Z}; 0, 1, +, V_k, \leq)$ is an ultimately periodic set.*

Lemma 3.10 implies Corollary 1.3 from Section 1.3, and shows that the decision problems of EXISTENCE and UNIVERSALITY for parametric Büchi arithmetic are decidable.

THEOREM 3.11. *The EXISTENCE and UNIVERSALITY problems for Büchi arithmetic are decidable.*

PROOF. For both problems, we first run Algorithm 2 on the input formula φ to compute a representation (B, N, p) of the ultimately periodic set $\mathcal{B}(\varphi)$. The formula φ is a positive instance of EXISTENCE if and only if B is non-empty. And it is a positive instance of UNIVERSALITY if and only if (B, N, p) represents $\mathbb{N}_{\geq 2}$, which holds whenever $N \geq 2$ and $B = [2..N + p - 1]$. $\square$

4 THE EFFECTS OF ADDING (FULL) MULTIPLICATION BY t

The reader may notice that in the proof of Theorem 1.2 we compared two subsequent configurations of parametric symbolic automata using a version of multiplication $(x \mapsto t \cdot x)$, restricted to inputs that are powers of t . In Büchi arithmetic on base k , the function $(x \mapsto k \cdot x)$ is clearly definable by a quantifier-free formula, and it is used extensively in the proof of Theorem 1.1, as well as in [Starchak 2023], where recognizable languages are characterized using existential formulas.

The latter paper shows that a set $S \subseteq \mathbb{N}^n$, being encoded in base k , is recognizable by a finite automaton if and only if it is existentially definable in the structure $(\mathbb{N}; 0, 1, +, \&_k, \leq)$, where $\&_k$ is the bitwise minimum operation on base k on non-negative integers:

$$(\langle 0a_\ell \dots a_0 \rangle_k \&_k \langle 0b_\ell \dots b_0 \rangle_k) = \langle 0 \min(a_\ell, b_\ell) \dots \min(a_0, b_0) \rangle_k. \quad (1)$$

Informally, the roles of $(x \mapsto k \cdot x)$ and $\&_k$ can be described as follows: the former is used to shift a representation of a sequence of configurations by one step, while the latter serves as a comparator of each pair of subsequent configurations. For Theorem 1.1, this means that in converting finite automata to Büchi arithmetic on base k , quantifier alternation is only needed to define $\&_k$.

In this section we study computational properties of parametric Büchi arithmetic enriched with $(x \mapsto t \cdot x)$, which we denote by $PBA[t \cdot]$. We start by considering a different signature, denoted $PBA[t, \&_t]$, which is obtained from $PBA[t \cdot]$ by replacing the predicate V_t with the binary function $\&_t$ to be interpreted as the bitwise minimum operation. We show that, in contrast to the case when the parameter t is replaced with a fixed integer $k \geq 2$, existential formulas over $PBA[t, \&_t]$ are expressive enough to encode computations of counter machines. This implies that the EXISTENCE problem for the existential fragment of $\mathcal{F}_{PBA[t, \&_t]}$ is already undecidable. Since the function $\&_t$ is expressible in $\mathcal{F}_{PBA[t \cdot]}$ using a formula with only universal quantifiers, this implies that the EXISTENCE problem for the $\exists^* \forall^*$ fragment of $\mathcal{F}_{PBA[t \cdot]}$ is undecidable, which in turn means (due to Theorem 3.11) that the function $(x \mapsto t \cdot x)$ is not definable in parametric Büchi arithmetic.

For simplicity, throughout the section we work with interpretations over the non-negative integers. Since PBA features the order $\leq$, our undecidability results trivially transfer to the integers.

4.1 The semantics of multiplication and bitwise minimum, and counter machines

The satisfaction relation $\models_k$ and semantics $\llbracket _ \rrbracket$ of parametric Büchi arithmetic can be naturally updated to the signatures $PBA[t \cdot]$ and $PBA[t, \&_t]$ and to work over $\mathbb{N}$. Given $k \in \mathbb{N}_{\geq 2}$, one can consider a single structure $\mathfrak{B}a_k^+ := (\mathbb{N}; I)$, where $0, 1, +$ and $\leq$ are interpreted over $\mathbb{N}$ in the usual way, the symbol V_t with the set $\{(n, d) \in \mathbb{N} \times \mathbb{N} : d \text{ is the largest power of } k \text{ dividing } n\}$, and the binary function $\&_t : \mathbb{N} \times \mathbb{N} \rightarrow \mathbb{N}$ with the bitwise minimum operation on base k , following the equation in (1). For $\mathcal{F} = \mathcal{F}_{PBA[t \cdot]}$ or $\mathcal{F} = \mathcal{F}_{PBA[t, \&_t]}$, this structure naturally yields a satisfaction relation $_ \models_k _ : [X \rightarrow \mathbb{N}] \rightarrow \mathcal{F} \rightarrow \{\top, \perp\}$, from which one can then define the (parametric) semantics $\llbracket _ \rrbracket : \mathcal{F} \rightarrow \mathbb{N}_{\geq 2} \rightarrow \mathcal{P}[X \rightarrow \mathbb{N}]$, exactly as done for parametric Büchi arithmetic. One can then define the set of realizable bases $\mathcal{B}(\varphi)$ and the EXISTENCE and UNIVERSALITY problems for φ in $\mathcal{F}_{PBA[t \cdot]}$ or $\mathcal{F}_{PBA[t, \&_t]}$, exactly as in Section 2.

Since we are now working over $\mathbb{N}$, some care is needed regarding subtraction. In this section, we solely consider the binary minus operator $- : \mathbb{N} \times \mathbb{N} \rightarrow \mathbb{N}$ given by $x - y = z$ if and only if $(z = 0 \wedge x < y) \vee (x = z + y)$. We will work as if this operator is available in the signature. It is easy to see that any formula featuring binary minus can be rewritten into an equivalent one without it, at the expenses of adding further existentially quantified variables.

Counter machines. A counter machine [Kozen 1997; Minsky 1967] is a tuple $\mathcal{M} := (Q, C, q_0, F, \Delta)$, where Q is a finite set of states, $C = \{c_1, \dots, c_n\}$ is a finite set of $n \geq 1$ counters, each storing a non-negative integer, $q_0 \in Q$ is the initial state, $F \subseteq Q$ are final states, and $\Delta \subseteq Q \times \{\text{id}, ++, --\}^n \times Q$ is the transition relation, where id , $++$, and $--$ are the identity, successor, and predecessor operations on integers, respectively. A *configuration* of $\mathcal{M}$ is a tuple $(p, c_1, \dots, c_n) \in Q \times \mathbb{N}^n$. We write $\rightsquigarrow$ for the smallest binary relation on configurations such that, for every $(p, c_1, \dots, c_n) \in Q \times \mathbb{N}^n$,

$$(p, c_1, \dots, c_n) \rightsquigarrow (q, f_1(c_1), \dots, f_n(c_n)) \iff \exists (p, f_1, \dots, f_n, q) \in \Delta \text{ s.t. } f_i(c_i) \geq 0 \text{ for all } i \in [1..n].$$

A *run* in $\mathcal{M}$ is a sequence of configurations $(p, c_1, \dots, c_n) \rightsquigarrow \dots \rightsquigarrow (p', c'_1, \dots, c'_n)$. We say that a counter machine $\mathcal{M}$ *accepts* $x \in \mathbb{N}$ whenever for some $q \in F$ there is a run in $\mathcal{M}$ of the form

($q_0, x, 0, \dots, 0$) $\rightsquigarrow \dots \rightsquigarrow (q, 0, 0, \dots, 0)$. The following classical result (see, e.g., [Kozen 1997]), implies, in particular, that the non-emptiness problem (i.e., does a given $\mathcal{M}$ accept any number) is undecidable already for counter machines with 3 counters.

FACT 4.1. *A set $S \subseteq \mathbb{N}$ is recursively enumerable if and only if there is a counter machine with 3 counters that accepts exactly the numbers in S .*

4.2 Encoding runs with existential formulas

Using counter machines, we now show how to encode any recursively-enumerable set $S \subseteq \mathbb{N}$ as an existential formula in $\mathcal{F}_{PBA[t, \&_t]}$. Other encoding of counter/register machines into purely existential formulas can be found, for instance, in [Jones and Matijasevič 1984] for the $\exists^*$ fragment of $(\mathbb{N}; 0, 1, +, \cdot, x^y, =)$ (i.e., the structure of exponential Diophantine equations) and in [Starchak 2023] for the $\exists^*$ fragment of $(\mathbb{N}; 0, 1, +, \&_k, \frown_k, \leq)$, where $\frown_k$ stands for concatenation of k -ary representations of non-negative integers. Though these two theories and $\mathcal{F}_{PBA[t, \&_t]}$ can represent all recursively-enumerable sets, we do not see any simple reduction between them.

THEOREM 4.1. *Let $S \subseteq \mathbb{N}$ be a recursively-enumerable set. There is a univariate existential formula $R(x)$ in $\mathcal{F}_{PBA[t, \&_t]}$ such that $S = \{a \in \mathbb{N} : \text{there is } k \geq 2 \text{ such that } [x \mapsto a] \models_k R(x)\}$.*

PROOF. We construct φ from a counter machine $\mathcal{M} = (Q, \{c_1, c_2, c_3\}, q_0, F, \Delta)$ accepting the numbers in S . Let us start by giving an intuition on how to encode accepting runs of $\mathcal{M}$ via variables over the non-negative integers. In an accepting run $(q_0, x, 0, 0) \rightsquigarrow \dots \rightsquigarrow (q, 0, 0, 0)$ with $q \in F$ and $x \in \mathbb{N}$, the values of the three counters are necessarily bounded by some non-negative integer, say M . If we force the parameter t to be greater than M (in fact $M + 2$ for technical reasons), then every value of the counters can be encoded with a single digit in base t . The idea is then to use variables y_1, y_2, y_3 to encode the values of the three counters in the run: the i -th digit of y_j in base t will be equal to the value of counter c_j in the i -th configuration of the run (plus 1, again for technical reasons). As in the proof of Lemma 3.8, to encode the sequence of states in the run, for each state $q \in Q$ we will introduce a non-negative integer variable with the same name, and require that the base t encoding of q belongs to $\{0, 1\}^*$, with that i -th digit equal to 1 if and only if the state in the i -th configuration of the run is q . An additional variable w is used to encode a power t^m , where $m + 1$ is the length of the run. Most of the work will then be to express the fact that the sequence of configurations encoded by these variables is indeed a valid and accepting run of $\mathcal{M}$.

Let us formalize the above intuition. To do so, we need a few auxiliary predicates, the first set of which is defined in the next table.

Predicate:	Definition:	Meaning:
$P_t(x)$	$1 \leq x \wedge x \&_t (x - 1) = 0$	x is a power of t .
$x \subseteq_t y$	$x \&_t y = x$	Every digit of x is smaller than or equal to the corresponding digit in y .
$x = \mathbf{1}_t(w)$	$P_t(w) \wedge (t - 1) \cdot x = t \cdot w - 1$	w is of the form t^m and the base t encoding of x is a sequence of $m + 1$ many 1s.

For example, we have $15 = \mathbf{1}_2(8)$ and $111 = \mathbf{1}_{10}(100)$. We will often use $x = \mathbf{1}_t(w)$ as a function $\mathbf{1}_t(w)$. Occurrences of this function can be expressed with the predicate by introducing fresh existential variables. We will also need to extract from two non-negative integers x and y the unique integer z filtering exactly the digits of x and y that equal: for every position $i \in \mathbb{N}$ we have $z[i] = \text{if } x[i] = y[i] \text{ then } x[i] \text{ else } 0$. This ternary predicate is denoted by $z = eq_t(x, y)$, and can be defined as the following $\exists^*$ formula of $\mathcal{F}_{PBA[t, \&_t]}$:

$$\exists v: P_t(v) \wedge (x < v) \wedge z = x \&_t ((t - 1) \cdot (\mathbf{1}_t(v) - (\mathbf{1}_t(v) \&_t ((x - x \&_t y) + (y - y \&_t x))))).$$

This definition executes a sequence of digit-wise transformations. The subformula $x - x \&_t y$ defines an integer z_x such that $z_x[i] = 0$ if and only if $x[i] \leq y[i]$; while $y - y \&_t x$ symmetrically yields an analogous integer z_y . Their sum $z_x + z_y$, corresponding to the expression within the orange parentheses, is an integer satisfying $(z_x + z_y)[i] = 0$ if and only if $x[i] = y[i]$. The next step applies a digit-wise indicator function to this sum, mapping non-zero digits to 1 and zeros to 0. This is achieved by using a sufficiently large power of t (the variable v) to generate a mask of 1s via $\mathbf{1}_t(v)$, which is then intersected with $z_x + z_y$ via the bitwise minimum (expression within the parentheses in light blue). Subtracting this sequence from $\mathbf{1}_t(v)$ (expression within the parentheses in blue) thus yields the unique integer r such that, for every position $i \in \mathbb{N}$, $r[i] = \text{if } x[i] = y[i] \text{ then } 1 \text{ else } 0$. Then, the target integer z is obtained by scaling r by $(t - 1)$ and taking its bitwise minimum with x .

Let us now consider the counter machine $\mathcal{M} = (Q, \{c_1, c_2, c_3\}, q_0, F, \Delta)$. As done in the proof of Lemma 3.8, for every state $q \in Q$ we introduce a variable with the same name, and denote these variables collectively as $\mathbf{q}$. To capture the sequence of states in an accepting run of length $m + 1$, we impose the following conditions:

- (1) for every $i \in [0..m]$ exactly one $q \in Q$ is such that $q[i] = 1$, the others have a zero digit;
- (2) $q_0[0] = 1$, and $q[j] = 1$ for some $j \in [0..m]$ and $q \in F$.

Considering a further variable w , used to encode the power t^m , we can capture these two properties with the following formula:

$$\text{run}_{Q,F}(w, \mathbf{q}) \coloneqq \underbrace{\sum_{q \in Q} q = \mathbf{1}_t(w) \wedge \bigwedge_{q \neq q' \in Q} q \&_t q' = 0 \wedge 1 \subseteq_t q_0 \wedge w \subseteq_t \sum_{q \in F} q}_{\text{Item 1}} \quad \underbrace{\phantom{\sum_{q \in Q} q = \mathbf{1}_t(w) \wedge \bigwedge_{q \neq q' \in Q} q \&_t q' = 0 \wedge 1 \subseteq_t q_0 \wedge w \subseteq_t \sum_{q \in F} q}}_{\text{Item 2}}$$

The values of the counters c_1, c_2, c_3 during the run are encoded by three variables y_1, y_2, y_3 . We will restrict the digits of these variables in base t to $[1..t-2]$. This is done for simplicity and technical reasons: avoiding the digit $t - 1$ ensures the additions we will perform do not produce a carry, while starting from 1 allows us to use 0 as a “missing value”. For every $j \in \{1, 2, 3\}$ and $i \in [0..m]$, the i -th digit of y_j in base t is equal to 1 plus the value of counter c_j in the i -th configuration of the run. Furthermore, since y_1, y_2, y_3 are used to encode the values of counters that untouched, increased by 1, or decreased by 1 at each step of the run, we also want to impose that the base t representation of $y \in \{y_1, y_2, y_3\}$ satisfies $y[i+1] \in \{y[i] - 1, y[i], y[i] + 1\}$ for every position such that for every $i \geq 0$. With this in mind, consider the formula:

$$C(w, y, v, u_{\text{id}}, u_{++}, u_{--}) \coloneqq (\mathbf{1}_t(w) \subseteq_t y) \wedge (y \subseteq_t (t - 2) \cdot \mathbf{1}_t(w)) \wedge (y = u_{\text{id}} + u_{++} + u_{--} + v) \wedge \quad (2)$$

$$(v \leq t - 2) \wedge (u_{\text{id}} \&_t u_{++} = 0) \wedge (u_{\text{id}} \&_t u_{--} = 0) \wedge (u_{++} \&_t u_{--} = 0) \wedge \quad (3)$$

$$u_{\text{id}} = \text{eq}_t(y, (t \cdot y)) \wedge u_{++} = \text{eq}_t(y, (t \cdot (y + \mathbf{1}_t(w)))) \wedge u_{--} = \text{eq}_t(y, (t \cdot (y - \mathbf{1}_t(w)))). \quad (4)$$

In this formula, w again encodes the power t^m . The first two conjuncts in (2) ensure that the digits of y are in $[1..t-2]$, while the last conjunct decomposes y into four distinct parts, stored in the free variables v, u_{id}, u_{++} , and u_{--} . Because of (3) and (4), this decomposition is disjoint: for every $i \in [0..m]$, exactly one of the four variables has a non-zero digit in position i . Indeed, observe that (among other things) the conjuncts in (4) ensure that the least-significant digits of u_{id}, u_{++} , and u_{--} are all equal to zero. Then, the conjuncts in (3) ensure that v has a single digit in base t (it is in fact the least significant digit of y), and that the other three variables have no overlapping non-zero digits. Looking at $u_{\text{id}} = \text{eq}_t(y, (t \cdot y))$, we see that u_{id} has a non-zero digit in position $i \geq 1$ if and only if $y[i] = y[i-1]$, and in this case $u_{\text{id}}[i] = y[i]$. Similarly, $u_{++} = \text{eq}_t(y, (t \cdot (y + \mathbf{1}_t(w))))$ and $u_{--} = \text{eq}_t(y, (t \cdot (y - \mathbf{1}_t(w))))$ ensure that u_{++} and u_{--} have a non-zero digit in position $i \geq 1$ if and only if $y[i] = y[i-1] + 1$ and $y[i] = y[i-1] - 1$, respectively, and in these cases $u_{++}[i] = y[i]$ and $u_{--}[i] = y[i]$. Note that, as each digit of y are in $[1..t-2]$, the addition and subtraction of $\mathbf{1}_t(w)$ in these formulas do not produce any carry.

EXAMPLE 4.1. Suppose that the values of a counter c follows the following progression during a run: 2, 1, 0, 0, 0, 1, 2, 1, 1, 0. This sequence will be represented in the base $t \geq 5$ encoding using the non-negative integer $y = (123211123)_t$. The formula $C(w, y, v, u_{id}, u_{++}, u_{--})$ forces v to be equal to the least significant digit of y , which is 3 in this example. This corresponds to the initial value of the counter c , plus 1. The subformula $\mathbf{1}_t(w) \subseteq_t y$ imposes $\mathbf{1}_t(w) = (111111111)_t$, so $w = t^9$ (the run has length 10). Together with v , the remaining integers in the formula decompose y as follows: $u_{id} = (200011000)_t$, $u_{++} = (3200000)_t$, and $u_{--} = (1020000120)_t$. ◀

Lastly, we define the formula $R(x)$ that expresses the existence of an accepting run of $\mathcal{M}$ on input x . Denoting $\mathbf{y} := (y_1, y_2, y_3)$ and $\mathbf{u} := (u_{id}^1, u_{++}^1, u_{--}^1, u_{id}^2, u_{++}^2, u_{--}^2, u_{id}^3, u_{++}^3, u_{--}^3)$, we define

$$R(x) := \exists w, \mathbf{y}, \mathbf{u}, \mathbf{q} \left(\text{run}_{Q,F}(w, \mathbf{q}) \wedge C(w, y_1, x + 1, u_{id}^1, u_{++}^1, u_{--}^1) \wedge \bigwedge_{i \in \{2,3\}} C(w, y_i, 1, u_{id}^i, u_{++}^i, u_{--}^i) \right. \\ \left. \wedge \bigwedge_{i \in \{1,2,3\}} (y_i < 2 \cdot w) \wedge \bigwedge_{(q, f_1, f_2, f_3) \in Q \times \{\text{id}, ++, --\}^3} (t \cdot q) \&_t u_{f_1}^1 \&_t u_{f_2}^2 \&_t u_{f_3}^3 \subseteq_t \sum_{\substack{q' \in Q: \\ (q, f_1, f_2, f_3, q') \in \Delta}} q' \right).$$

We now verify that $R(x)$ correctly encodes an accepting run.

Both the $\text{run}_{Q,F}(w, \mathbf{q})$ and $C(\dots)$ formulas use w within the $\mathbf{1}_t(\cdot)$ function, specifically in the expressions $\sum_{q \in Q} q = \mathbf{1}_t(w)$ and $(\mathbf{1}_t(w) \subseteq_t y) \wedge (y \subseteq_t (t - 2) \cdot \mathbf{1}_t(w))$. These constraints force y_1, y_2, y_3 and $\sum_{q \in Q} q$ to all have the same length $m + 1$ in base t , which determines that $w = t^m$. The length $m + 1$ thus represents the number of configurations in the encoded run.

Recall that in $C(w, y, v, u_{id}, u_{++}, u_{--})$, the parameter v captures the least significant digit of y . In $R(x)$, we instantiate C with $v = x + 1$ for the first counter and $v = 1$ for the remaining counters, thereby initializing the three counters to $(x, 0, 0)$ for the initial configuration of the run. The formula C also decomposes each subsequent digit of y_i based on whether the counter remains unchanged (u_{id}^i), increments (u_{++}^i), or decrements (u_{--}^i). For the run to be accepting, the final configuration must have all counters at zero. Since counter values are shifted by $+1$ in our encoding, this means each y_i must have most significant digit equal to 1, which is equivalent to requiring $y_i < 2 \cdot w$.

The formula $\text{run}_{Q,F}(w, \mathbf{q})$ ensures that the encoded run visits exactly one state at each step, starting from the initial state q_0 and ending in a final state $q \in F$. The remaining task is to encode the transition relation Δ , which is done by the final conjunction in $R(x)$.

At each step $i \in [0..m]$, exactly one state variable $q \in Q$ satisfies $q[i] = 1$. Similarly, for the transition from step i to step $i + 1$ (where $i \in [0..m - 1]$), exactly one triple $(f_1, f_2, f_3) \in \{\text{id}, ++, --\}^3$ of counter operations is applied, meaning $u_{f_1}^1[i + 1]$, $u_{f_2}^2[i + 1]$, and $u_{f_3}^3[i + 1]$ are all non-zero. To verify that the transition from step i to step $i + 1$ respects Δ , we use multiplication by t to shift the state information: since $(t \cdot q)[i + 1] = q[i]$, the term $(t \cdot q) \&_t u_{f_1}^1 \&_t u_{f_2}^2 \&_t u_{f_3}^3$ isolates exactly those positions $i + 1$ where state q is active at step i and operations (f_1, f_2, f_3) are applied from step i to step $i + 1$. For the run to be valid, the successor state at step $i + 1$ must be reachable via this quadruple. The condition $(t \cdot q) \&_t u_{f_1}^1 \&_t u_{f_2}^2 \&_t u_{f_3}^3 \subseteq_t \sum_{q' \in Q: (q, f_1, f_2, f_3, q') \in \Delta} q'$ enforces this: wherever the left-hand side has a non-zero digit (indicating that state q and operations (f_1, f_2, f_3) are active), the right-hand side must also have a non-zero digit, indicating that at least one valid successor state $q' \in Q$ exists according to Δ . ◻

Theorem 4.1 and the undecidability of the non-emptiness problem for counter machines imply:

COROLLARY 4.2. *The EXISTENCE problem for the $\exists^*$ fragment of $\mathcal{F}_{PBA[t, \&_t]}$ is undecidable.*

With Theorem 3.11, the following corollary implies Theorem 1.4 from Section 1.3.

COROLLARY 4.3. *The EXISTENCE problem for the $\exists^* \forall^*$ fragment of $\mathcal{F}_{PBA[t, \cdot]}$ is undecidable.*

PROOF. Because of Corollary 4.2, it suffices to show how to express any existential formula of $\mathcal{F}_{PBA[t, \&_t]}$ as a formula in the $\exists^* \forall^*$ fragment of $\mathcal{F}_{PBA[t, \cdot]}$.

Let $\exists x \varphi(x)$ be an existential formula in $\mathcal{F}_{PBA[t, \&_t]}$, with φ quantifier-free. Applying the function

$$\Psi := \psi \mapsto \begin{cases} \psi & \text{if } \psi \text{ does not contain } \&_t, \\ \exists z \left(z = \tau \&_t \rho \wedge \Psi(\psi[z/\tau \&_t \rho]) \right) & \text{if } \psi \text{ contains } \tau \&_t \rho, \text{ with } \tau, \rho \text{ terms from } PBA[t, \cdot] \end{cases}$$

on φ , we translate $\exists x \varphi(x)$ into an equivalent formula $\exists x \Psi(\varphi)$ in which $\&_t$ occurs only in atomic formulas of the form $z = \tau \&_t \rho$, and never in the scope of a negation. To prove the corollary it thus suffices to express $z = x \&_t y$ using a formula in $\mathcal{F}_{PBA[t, \cdot]}$ with only universal quantifiers.

In our definition, we use the quantifier-free formula $\text{split}_t(x, p, p', p'', x_1, x_2, x_3)$ from Lemma 3.6. Recall that, assuming that p and p' are powers of t such that $p' = t \cdot p$, the variable x is uniquely decomposed into $x_1 < p$, $x_2 = d \cdot p$ for some digit $d \in [0..t-1]$, and x_3 is divisible by $p'' \geq p'$. It remains to compare the digits in x, y , and z using universal quantifiers.

$$\begin{aligned} z = x \&_t y \leftrightarrow \forall p, \mathbf{p}, \mathbf{x}, \mathbf{y}, \mathbf{z} \left(\text{split}_t(x, p, t \cdot p, p_x, \mathbf{x}) \wedge \text{split}_t(y, p, t \cdot p, p_y, \mathbf{y}) \wedge \text{split}_t(z, p, t \cdot p, p_z, \mathbf{z}) \right. \\ \left. \rightarrow (z_2 = x_2 \wedge x_2 \leq y_2) \vee (z_2 = y_2 \wedge y_2 \leq x_2) \right), \end{aligned}$$

where $\mathbf{p} := (p_x, p_y, p_z)$, and the triples of variables in the decomposition of x, y, z are gathered into $\mathbf{x} := (x_1, x_2, x_3)$, $\mathbf{y} := (y_1, y_2, y_3)$, and $\mathbf{z} := (z_1, z_2, z_3)$. The second variable in each triple is responsible for the digit base t on the i -th position for $p = t^i$, and we take the minimum of x_2 and y_2 . $\square$

5 PRACTICAL CONSIDERATIONS

In the published literature, much effort has been put into designing and implementing optimized translations to automata (see, e.g., [Boigelot 2021; Habermehl et al. 2024; Nicol and Frohme 2026; Oei et al. 2021] for recent efforts), as well as into understanding what types of arithmetical and combinatorial problems can be solved with Büchi arithmetic [Hieronymi et al. 2024; Popoli et al. 2024; Shallit 2022]. This section is similar in spirit. In Section 5.1 we provide an optimization on the translation of linear inequalities into parametric symbolic automata given in Section 3.2. In Sections 5.2 and 5.3 we discuss a few arithmetical problems from the literature which can be expressed as bounded existence and universality problems for quantifier-free PBA and describe our experiments with a prototype tool, on benchmarks featuring these arithmetical problems.

5.1 Translating inequalities

Translating inequalities $\sum_{i=1}^n a_i \cdot x_i \leq c$ by combining the “base cases” as in Section 3.2 is extremely inefficient in practice. For Büchi arithmetic in base k , Algorithms 1 and 2 from [Boigelot 2021] translate equations and inequalities into automata with $O(k \cdot \log |b| \cdot \|\mathbf{a}\|_1)$ states, where $\mathbf{a} = (a_1, \dots, a_n)$ and $\|\mathbf{a}\|_1 := \sum_{i=1}^n |a_i|$. For equations over $\mathbb{N}$, [Guépin et al. 2019, Proposition 5] gives a base-independent translation with $O(\max(\|\mathbf{a}\|_1, |b|))$ states.

We present a variant of the algorithm from [Boigelot 2021] for parametric symbolic automata, together with a novel complexity analysis, showing that inequalities can be translated into parametric symbolic automata with $O(\sqrt{|b|} \cdot \|\mathbf{a}\|_1 + \|\mathbf{a}\|_1)$ states. Asymptotically, this bound is better than in [Guépin et al. 2019], and (in addition to being base-independent) it improves on [Boigelot 2021] by a logarithmic factor whenever $|b| \leq \|\mathbf{a}\|_1$. The pseudocode is given in Algorithm 3.

As a preliminary step, the algorithm normalizes the coefficients by dividing them by their gcd g (lines 1–2), using the equivalence $\sum_{i=1}^n a_i \cdot x_i \leq b \leftrightarrow \sum_{i=1}^n (a_i/g) \cdot x_i \leq \lfloor b/g \rfloor$. This step, absent from [Boigelot 2021], simplifies both the definition of the state set Q and the complexity analysis.

Algorithm 3 Builds the parametric symbolic automaton of an inequality.

Input: An inequality $\varphi := \sum_{i=1}^n a_i \cdot x_i \leq b$ with $a_i, b \in \mathbb{Z}$

Output: A valid NSA $\mathcal{A} \in \text{Aut}(PA\mathcal{I})$ such that $\mathcal{L}_{\text{msdf}}(\mathcal{A}) = \llbracket \varphi \rrbracket$

- 1: **let** $g := \gcd(a_1, \dots, a_n)$
 - 2: $(a_1, \dots, a_n, b) \leftarrow (a_1/g, \dots, a_n/g, \lfloor b/g \rfloor)$
 - 3: **let** $a_+ := \sum_{a_i > 0} a_i$ and $a_- := \sum_{a_i < 0} a_i$
 - 4: **let** $Q := \{\lfloor b/k \rfloor + \ell : k \in [1..|b|+1], \ell \in [-a_+..-a_-]\}$ and $F := \{b\}$
 - 5: **let** q_0 be a fresh state, and let the transition function δ be defined as follows:

$$(v_1, v_2) \mapsto (v_1 \cdot t \leq (v_2 - \sum_{i=1}^n a_i \cdot x_i) < (v_1 + 1) \cdot t) \quad \text{for all } v_1 \in Q, v_2 \in Q$$

$$(q_0, v) \mapsto \sum_{i=1}^n a_i \cdot x_i \geq v \cdot (1 - t) \quad \text{for all } v \in Q$$

$$(q, q_0) \mapsto \perp \quad \text{for all } q \in Q \cup \{q_0\}$$
 - 6: **return** $(\text{Im}(\delta), Q \cup \{q_0\}, q_0, F, \delta)$
-

The idea behind the subsequent steps is to identify each non-initial state of the resulting automaton with an integer v corresponding to the right-hand side of the inequality $\sum_{i=1}^n a_i \cdot \langle \mathbf{w} \rangle_k(x_i) \leq v$. More precisely, we seek for the following semantics for v : for any base $k \in \mathbb{N}_{\geq 2}$, a word $\mathbf{w} \in \mathcal{M}_k$ (recall: $\mathcal{M}_k := [X \rightarrow \{0, k-1\}] \cdot [X \rightarrow [0..k-1]]^*$) reaches v from the initial state if and only if $\sum_{i=1}^n a_i \cdot \langle \mathbf{w} \rangle_k(x_i) \leq v$. A priori, it is not clear why such a set of integers v should be finite and independent of the base k . Both properties stem from the following observation.

Assume for simplicity $b \geq 0$, and define the integers $a_+ := \sum_{a_i > 0} a_i$ and $a_- := \sum_{a_i < 0} a_i$. Take a word $\mathbf{w} = w_\ell \dots w_0 \in \mathcal{M}_k$ with $\ell \geq 1$ and let $\mathbf{w}' = w_\ell \dots w_1$. By definition of $\langle \cdot \rangle_k$, we have

$$\sum_{i=1}^n a_i \cdot \langle \mathbf{w} \rangle_k(x_i) \leq b \text{ if and only if } \sum_{i=1}^n a_i \cdot \langle \mathbf{w}' \rangle_k(x_i) \leq \left\lfloor \frac{b - \sum_{i=1}^n a_i \cdot w_0(x_i)}{k} \right\rfloor. \quad (5)$$

Since the image $\text{Im}(w_0)$ is included in $[0..k-1]$, we see that

$$a_- \cdot (k-1) \leq \sum_{i=1}^n a_i \cdot w_0(x_i) \leq a_+ \cdot (k-1).$$

Together with $0 \leq \frac{b}{k} \leq b$, this means that the floor division in the right-hand side $\left\lfloor \frac{b - \sum_{i=1}^n a_i \cdot w_0(x_i)}{k} \right\rfloor$ “transforms” b into an integer in the interval $[-a_+..b - a_-]$. This interval does not depend on the base k , on the length ℓ , and on the map w_ℓ . Hence, to recognize the inequality $\sum_{i=1}^n a_i \cdot \langle \mathbf{w} \rangle_k(x_i) \leq b$, it is sufficient for the automaton to track only integers from this finite set, and take $F = \{b\}$ as the only accepting state. The set of states Q defined in line 4 of Algorithm 3 follows this idea, but is slightly more refined in order to obtain a better bound on the number of states of the automaton.

LEMMA 5.1. *Let Q be the set defined in line 4 of Algorithm 3, with respect to the integers $a_1, \dots, a_n, b$ in line 2. Then, (1) $b \in Q$, and (2) $\left\lfloor \frac{v - \sum_{i=1}^n a_i \cdot w(x_i)}{k} \right\rfloor \in Q$ for all $v \in Q, k \in \mathbb{N}_{\geq 2}$ and $w: X \rightarrow [0..k-1]$.*

Let us now examine the transitions. Since states in Q encode right-hand sides of inequalities, the transitions between them follow directly from (5): a pair $(v_1, v_2) \in Q \times Q$ must be labelled by a formula equivalent to $v_1 = \left\lfloor \frac{v_2 - \sum_{i=1}^n a_i \cdot x_i}{t} \right\rfloor$. Line 5 provides exactly this formula, expressed as the pair of linear inequalities $v_1 \cdot t \leq v_2 - \sum_{i=1}^n a_i \cdot x_i < (v_1 + 1) \cdot t$.

It remains to discuss the transitions from the initial state $q_0 \notin Q$ (the reverse transitions, to q_0 , are all labelled $\perp$). These handle single-letter words $w \in [X \rightarrow \{0, k-1\}]$; recall that our goal in this algorithm is to express the fact that $\mathbf{w}$ reach $v \in Q$ from q_0 if and only if $\sum_{i=1}^n a_i \cdot \langle \mathbf{w} \rangle_k(x_i) \leq v$. By definition of the map $\langle \cdot \rangle_k$, $\langle \mathbf{w} \rangle_k(x) = 0$ when $w(x) = 0$ and $\langle \mathbf{w} \rangle_k(x) = -1$ when $w(x) = k-1$, hence $\langle \mathbf{w} \rangle_k(x) = \frac{w(x)}{1-k}$ for all $x \in X$. Thus, $\sum_{i=1}^n a_i \cdot \langle \mathbf{w} \rangle_k(x_i) \leq v$ rewrites as $\sum_{i=1}^n a_i \cdot w(x_i) \geq v \cdot (1-k)$, which is precisely the formula labelling the pair (q_0, v) in line 5.

LEMMA 5.2. *Algorithm 3 respects its specification.*

PROOF IDEA. Consider an inequality $\varphi := \sum_{i=1}^n a_i \cdot x_i \leq b$, assumed to satisfy $\gcd(a_1, \dots, a_n) = 1$ thanks to lines 1 and 2 of Algorithm 3. Let $\mathcal{A} = (\Phi, Q \cup \{q_0\}, q_0, F, \delta)$ be the automaton for φ constructed by Algorithm 3. To prove the lemma, one shows that for every $k \in \mathbb{N}_{\geq 2}$ we have $\mathcal{L}_{\text{msdf}}(\mathcal{A})(k) = \llbracket \varphi \rrbracket_k$, and that $\mathcal{A}$ is a valid NSA. For this, it suffices to consider $k \in \mathbb{N}_{\geq 2}$, a word $\mathbf{w} := w_\ell \dots w_0 \in \mathcal{M}_k$, and $v \in Q \subseteq \mathbb{Z}$. A simple induction on ℓ shows that $\sum_{i=1}^n a_i \cdot \langle \mathbf{w} \rangle_k(x_i) \leq v$ if and only if there is a path $q_0 \dots q_{\ell+1}$ such that $q_{\ell+1} = v$ and $w_{\ell+1-i} \models_k \delta(q_{i-1}, q_i)$ for every $i \in [1.. \ell + 1]$. As $b \in F$ (line 4), taking $v = b$ proves the lemma. $\square$

Complexity-wise, to show that Algorithm 3 computes an NSA with $O(\sqrt{|b| \cdot \|\mathbf{a}\|_1} + \|\mathbf{a}\|_1)$ states, it suffices to bound the size of the set Q from line 4. The following lemma provides such a bound.

LEMMA 5.3. *For any integers a, b, c with $a \leq c$, the number of integers in the Minkowski sum $[a..c] + \{\lfloor b/k \rfloor : k \in [1.. \lfloor b \rfloor + 1]\}$ is at most $2 \cdot \sqrt{|b| \cdot (c - a + 1)} + c - a + 1$.*

PROOF. Recall that Minkowski sum $A + B$ of two sets $A, B \subseteq \mathbb{Z}$ is just $\{a + b : a \in A \text{ and } b \in B\}$. For simplicity, assume $b \geq 0$; the case $b < 0$ is symmetric. The key observation is that as k grows, the intervals $I_k := [a..c] + \lfloor b/k \rfloor$ begin to overlap. This is already visible at the tail: $I_b \cup I_{b+1} = ([a..c] + 1) \cup [a..c] = [a..c+1]$. The idea is then to find an integer k_0 such that $\bigcup_{k=k_0}^{b+1} I_k$ is, informally, nearly a single interval, whereas $I_1, \dots, I_{k_0-1}$ are distinct enough. We take $k_0 := \sqrt{\frac{b}{c-a+1}}$ as a “good enough” cut-off point and finish the proof as follows.

First, consider $k \geq k_0$. The set $\{\lfloor b/k \rfloor : k \in [\lceil k_0 \rceil .. b + 1]\}$ is included in the real interval $[0, b/k_0] = [0, \sqrt{b \cdot (c - a + 1)}]$. Therefore,

$$[a..c] + \{\lfloor b/k \rfloor : k \in [\lceil k_0 \rceil .. b + 1]\} \subseteq [a, c + \sqrt{b \cdot (c - a + 1)}],$$

and thus this part of the sum contributes at most $\sqrt{b \cdot (c - a + 1)} + c - a + 1$ integer points.

Consider now $k < k_0$. The integers in $\{\lfloor b/k \rfloor : k \in [1.. \lceil k_0 \rceil - 1]\}$ are all “sparse” but very few, and so a crude estimate suffices. The set $[a..c] + \{\lfloor b/k \rfloor : k \in [1.. \lceil k_0 \rceil - 1]\}$ has at most $k_0 \cdot (c - a + 1) \leq \sqrt{b \cdot (c - a + 1)}$ integer points.

Hence, $[a..c] + \{\lfloor b/k \rfloor : k \in [1.. b + 1]\}$ has size at most $2 \cdot \sqrt{b \cdot (c - a + 1)} + c - a + 1$. $\square$

REMARK 5.1. *Modifying Algorithm 3 so that it translates equalities $\sum_{i=1}^n a_i \cdot x_i = b$ into valid NSAs is straightforward. First, if b is not divisible by $\gcd(a_1, \dots, a_n)$, we have the algorithm return the an automaton for the empty language. Otherwise, it suffices to update the transition function as follows:*

$$(v_1, v_2) \mapsto v_1 \cdot t = v_2 - \sum_{i=1}^n a_i \cdot x_i \quad \text{for all } v_1 \in Q, v_2 \in Q$$

$$(q_0, v) \mapsto \sum_{i=1}^n a_i \cdot x_i = v \cdot (1 - t) \quad \text{for all } v \in Q$$

Correctness follows by minor adjustments to the proof of Lemma 5.2, since the equality analogue of the double implication in (5) is simply $\sum_{i=1}^n a_i \cdot \langle \mathbf{w} \rangle_k(x_i) = b \iff \sum_{i=1}^n a_i \cdot \langle \mathbf{w}' \rangle_k(x_i) = \frac{b - \sum_{i=1}^n a_i \cdot w_0(x_i)}{k}$.

5.2 SMT formalization of bounded arithmetical problems

Parametric Büchi arithmetic enables simultaneous reasoning about multiple bases, allowing us to express many natural arithmetical problems. In this section, we present two examples of such problems and formalize them as the BOUNDED EXISTENCE and BOUNDED UNIVERSALITY problems. For an input formula φ and $\alpha, \beta \in \mathbb{N}_{\geq 2}$, the question is whether $[\alpha.. \beta] \cap \mathcal{B}(\varphi) \neq \emptyset$ (resp. $[\alpha.. \beta] \subseteq \mathcal{B}(\varphi)$).

Moving towards our implementation, which we cover in the next section, we formalize these problems in the syntax our tool implements. This is the syntax of quantifier-free formulas of LIA with Strings (QF_SLIA) of SMT-LIB [Barrett et al. 2016], enriched with $\text{int} \cdot v : \mathbf{Int} \times \mathbf{Int} \rightarrow \mathbf{Bool}$

(corresponding to V_t). For simplicity, we still call this logic QF_SLIA. Variables are restricted to the sorts **Int** and **String**, and one can use the function $\text{str.to_int} : \mathbf{String} \rightarrow \mathbf{Int}$ for string-to-integer conversions (in base t). The logic features the standard arithmetic operations and comparisons on integers from QF_LIA, plus operations to build regular expressions (**RegEx**) over the alphabet $\{0, 1\}$, as for instance str.to_re to construct a regular expression from a 0-1 string, re.++ for concatenation, and re.* for Kleene star. Membership of a variable of sort **String** in a regular expression can be expressed using the predicate $\text{str.in_re} : \mathbf{String} \times \mathbf{RegEx} \rightarrow \mathbf{Bool}$.

Bounded local divisibilities. Our first example, already mentioned in Section 1.2, comes from the algorithms to decide the existential fragment of LIA with divisibility constraints [Bel'tyukov 1976; Lipshitz 1978]. Without entering into the details of these algorithms, it suffices to say that they perform several manipulations of the original formula, until reaching a system of inequalities

$$\bigwedge_{i=1}^n v_p(f_i(\mathbf{x})) \leq v_p(g_i(\mathbf{x})) \wedge f_i(\mathbf{x}) \neq 0 \wedge g_i(\mathbf{x}) \neq 0, \quad (6)$$

where p is a prime number, f_i and g_i are linear polynomials with integer coefficients, $\mathbf{x}$ are integer variables, and v_p is the p -adic valuation $v_p(n) := \max\{k \in \mathbb{N} : p^k \text{ divides } n\}$ (with $v_p(0) := +\infty$). The algorithm guarantees that the original formula from LIA with divisibility constraints is satisfiable if and only if this system has a solution for all primes p in some range $[\alpha.. \beta]$, and that one can equivalently look at all integers in this range instead of just the primes. This is a BOUNDED UNIVERSALITY problem, which we call the *bounded local divisibilities problem*.

Since $V_p(n, d) \leftrightarrow d = p^{v_p(n)}$ for all non-zero $n, d \in \mathbb{Z}$, the inequality $v_p(f_i(\mathbf{x})) \leq v_p(g_i(\mathbf{x}))$ in System (6) can alternatively be expressed using V_p . It is then straightforward to express System (6) in the syntax of our tool. For example, for $f(x, y) := 3 \cdot x - 6 \cdot y + 1$ and $g(x, y) := x - 1$, the formula $v_p(f) \leq v_p(g) \wedge f \neq 0 \wedge g \neq 0$ can be expressed as

$$(\text{and} (\text{int.v} (+ (* 3 x) (- (* 6 y)) 1) z) (\text{int.v} (- x 1) w) (<= z w))$$

where z and w are auxiliary integer variables. The constraints $f \neq 0$ and $g \neq 0$ are implied by the fact that $V_t(n, d)$ does not hold for $n = 0$.

Bounded modular polynomial roots. Our second example is a classic problem from number theory: given a polynomial $f(t)$ with integer coefficients, a modulus $m \in \mathbb{N}_{\geq 2}$, and two integers $\alpha, \beta \in \mathbb{Z}$, does there exist an integer $t \in [\alpha.. \beta]$ such that $f(t) \equiv 0 \pmod m$? Since we are working modulo m , α and β can be shifted so that they are both positive, and $\alpha \geq 2$. This problem is NP-complete already for f being a quadratic polynomial [Manders and Adleman 1976].

Taking for example $f(t) = 31 \cdot t^2 + 17 \cdot t + 48$, and $m = 96$, we can represent $f(t) \equiv 0 \pmod m$ in the syntax of our tool as follows:

$$\begin{aligned} &(\text{and} (\text{str.in_re } t1 (\text{re.++} (\text{re.*} (\text{str.to_re } "0")) (\text{str.to_re } "10")))) \\ &(\text{str.in_re } t2 (\text{re.++} (\text{re.*} (\text{str.to_re } "0")) (\text{str.to_re } "100")))) \\ & (= (+ (* 31 (\text{str.to_int } t2)) (+ (* 17 (\text{str.to_int } t1)) 48)) (* 96 q))) \end{aligned}$$

where $t1$, $t2$ and q are auxiliary variables. Notably, we use the base t of parametric Büchi arithmetic as the variable of the polynomial f . The formula encodes this by requiring $t1$ and $t2$ to have base t representations 0^*10 and 0^*100 respectively, which correspond to t and t^2 . The variable q is used to express the quotient of the division of $f(t)$ by m .

REMARK 5.2. *Of course, the algorithms tailored to the problems above would perform much better than reducing the problems to PBA and using a solver for it. The point of this section is to show that parametric Büchi arithmetic gives a general framework to express natural arithmetical problems.*

5.3 Experiments on bounded existence and universality problems

Solving BOUNDED EXISTENCE and BOUNDED UNIVERSALITY problems does not strictly require parametric Büchi arithmetic; standard Büchi arithmetic suffices by simply iterating through the bases in the range $[\alpha..\beta]$. This raises a pragmatic question: *Is there any benefit to using the parametric approach for bounded problems?* It is a priori unclear whether the answer is positive or negative. On the one hand, a parametric automaton must encode the behavior of the formula across all bases, making it larger than any single-base finite automaton. On the other hand, parametric automaton is constructed only once for all bases in $[\alpha..\beta]$. We investigate this question with our tool.

Our tool is a prototype implementation of the algorithms presented in this paper. Written in OCaml, it relies on the `smt.ml` (version 0.28.0) backend [Pereira et al. 2026] to call the SMT solver Z3 [de Moura and Bjørner 2008] (version 4.16.0) for solving the satisfiability of Presburger arithmetic formulas with digits, and to extract the realizable bases. Input formulas follow the syntax of QF_SLIA, as described in Section 5.2. The formula is preprocessed in a few ways, including putting it in negation normal form and replacing negated inequalities $\neg(f \leq g)$ by $g + 1 \leq f$, to avoid performing complementation on non-atomic formulas. The solver has two operating modes:

- `par` (short for *parametric mode*): This mode uses the parametric symbolic automata described in the paper. It constructs a single automaton bottom-up for a given formula.
- `sym` (short for *symbolic automata mode*): This mode uses symbolic automata for fixed bases: edge labels are Presburger formulas, but they feature the fixed base instead of the parameter t . On bounded problems, this mode iteratively constructs automata for each fixed $k \in [\alpha..\beta]$, e.g. using a symbolic variant of [Boigelot 2021, Algorithms 1 and 2] for (in)equalities.

In both modes, to prevent excessive memory consumption, we employ a heuristic that estimates the number of states in the bottom-up construction of the resulting automaton for a Boolean combination of atomic constraints. If it exceeds 500 000 states (this threshold is configurable via the `-active` option), instead of flattening these combinations into a single automaton, the solver perform an on-the-fly / lazy product construction similar to [Esparza and Blondin 2023, Algorithm 9]. In both active and lazy modes, we use depth-first search, and for every current state $q \in Q$, combine all labels of the outgoing transitions into a disjunction of PAt formulas, supplement the disjunction with the digit semantics for each digit variable, and employ Z3 to extract a successor of the state q , if the formula is satisfiable. The process terminates as soon as a path to an accepting state is found (or enough paths are found, in the case of BOUNDED UNIVERSALITY). We omit further details.

The benchmarks. We generated a set of 120 instances: 30 for the bounded local divisibilities, 30 for the bounded modular polynomial roots, and 60 random instances. Apart from fixing the “shape” of the formulas in the first two categories, all instances are random. Here are some statistics:

- Bounded local divisibilities: 1–3 comparisons of V_t , 3–6 variables, integers in $[-15..15]$.
- Bounded modular polynomial roots: degree of the polynomial in 1–4, modulus $m \in [19..1024]$ (because of some preprocessing, other integers are reduced by the tool modulo m).
- Random instances: 3–6 variables, 1–3 arithmetic constraints, 1–3 regular constraints, 1–2 occurrences of the `int.v` predicate, integers in $[-20..20]$.

As for the bases, we consider three intervals: $[2..20]$, $[21..200]$ and $[201..220]$. These intervals are chosen to perform three tests. (i) The first interval tests whether `sym` mode outperforms `par` mode on small intervals, where constructing a few small automata could be faster than constructing a single parametric automaton. (ii) The second interval tests whether `par` mode becomes faster for larger ranges. (iii) The third interval examines how base size affects performance in small ranges.

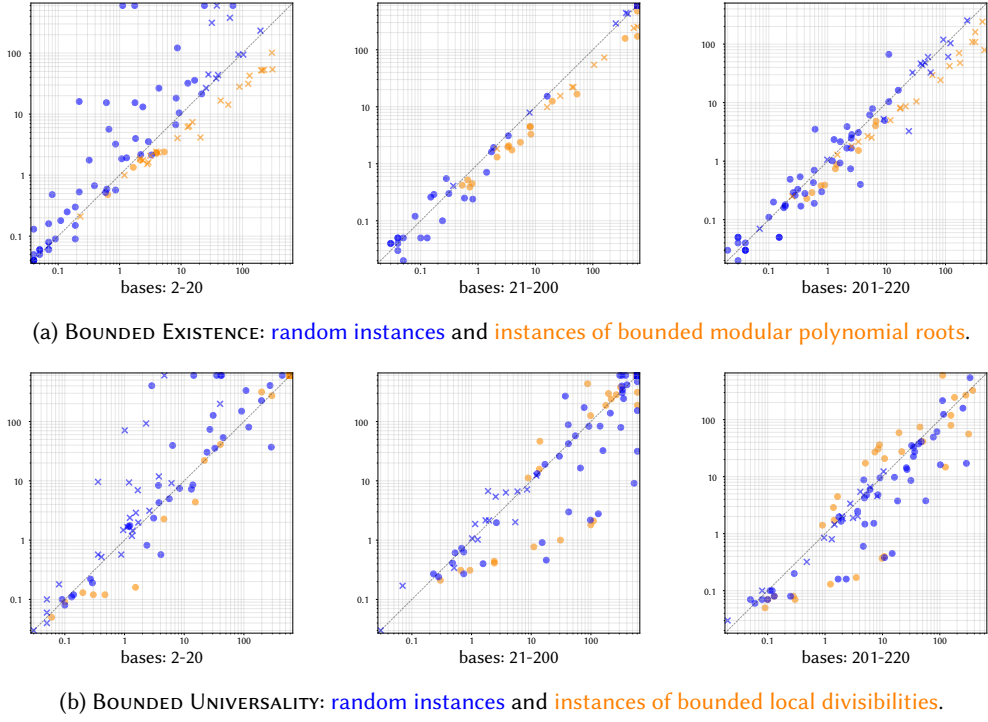


Fig. 4. Scatter plots for BOUNDED EXISTENCE and BOUNDED UNIVERSALITY problems. Circles represent SAT instances, while crosses represent UNSAT instances. The x- and y-axes show the execution time in seconds for the sym and par modes of the solver, respectively. Plots are in logscale; the time limit is set to 600 seconds.

Results. The results of our benchmarks are presented using scatter plots in Figure 4. Regarding the three tests above, the results seem to indicate that (i) sym mode is faster on intervals $[\alpha.. \beta]$ where β is small, (ii) par mode is faster on large intervals, and (iii) the performance of sym mode degrades for large bases, with par becoming faster on $[\alpha.. \beta]$ with α large. For example, for BOUNDED EXISTENCE problems with bases in the range $[201..220]$, par solves the 90 instances of the benchmark with a cumulative time of 1 931 seconds, against 3 488 seconds for sym mode.² Another interesting observation is that par mode is also consistently faster than sym mode on instances of the bounded modular polynomial root problem. This also holds for bases in the range $[2..20]$, although we note that most instances turned out to be UNSAT, which in retrospect is expected when the range of bases is small relative to the modulus. This might indicate that, in general, for instances that require exploring (almost) the whole range of bases, i.e. negative instances for BOUNDED EXISTENCE and positive instances for BOUNDED UNIVERSALITY, par mode is more efficient than sym mode.

6 CONCLUSION

We have introduced parametric symbolic automata that allow reasoning about integers simultaneously in different, potentially infinitely many positional encodings. Its arithmetical counterpart $\mathcal{F}_{PBA}$ provides a language that completely captures the expressive power of parametric symbolic automata. Using this language, we have proved decidability of the existence and universality problems for the standard Büchi arithmetic, and have shown that natural arithmetical problems featuring non-linear constraints can be modeled as parametric symbolic automata.

²Due to lack of space, further statistics are provided in the Jupyter notebook included as supplementary material.

REFERENCES

- Jean-Paul Allouche and Jeffrey O. Shallit. 2003. *Automatic Sequences - Theory, Applications, Generalizations*. Cambridge University Press.
- Nicolas Amat, Pierre Ganty, and Alessio Mansutti. 2025. How Big is the Automaton? Certified Lower Bounds on the Size of Presburger DFAs. In *ASE*. <https://doi.org/10.1109/ASE63991.2025.00219>
- Clark Barrett, Pascal Fontaine, and Cesare Tinelli. 2016. The Satisfiability Modulo Theories Library (SMT-LIB). www.SMT-LIB.org.
- Anatoly P. Bel'tyukov. 1976. Decidability of the universal theory of natural numbers with addition and divisibility. *Zap. Nauchn. Sem. LOMI* 60 (1976), 15–28. <https://doi.org/10.1007/BF01693974> (In Russian) English translation: in *J. Soviet Math.*, 14:5 (1980), 1436–1444.
- Siddharth Bhat, Léo Stefanescu, Chris Hughes, and Tobias Grosser. 2025. Certified Decision Procedures for Width-Independent Bitvector Predicates. *Proc. ACM Program. Lang.* OOPSLA (2025). <https://doi.org/10.1145/3763148>
- Tristram Bogart, John Goodrick, Danny Nguyen, and Kevin Woods. 2019. Parametric Presburger arithmetic: complexity of counting and quantifier elimination. *Math. Logic Quart.* (2019). <https://doi.org/10.1002/malq.201800068>
- Bernard Boigelot. 2021. Symbolic methods and automata. In *Handbook of Automata Theory*. <https://doi.org/10.4171/AUTOMATA-2/10>
- Alexandre Boudet and Hubert Comon. 1996. Diophantine equations, Presburger arithmetic and finite automata. https://doi.org/10.1007/3-540-61064-2_27
- Marius Bozga and Radu Iosif. 2005. On Decidability Within the Arithmetic of Addition and Divisibility. In *FoSSaCS*. https://doi.org/10.1007/978-3-540-31982-5_27
- Véronique Bruyère. 1985. Entiers et automates finis. (1985). Mémoire de fin d'études, University of Mons, Belgium.
- Véronique Bruyère and Georges Hansel. 1997. Bertrand Numeration Systems and Recognizability. *Theor. Comput. Sci.* (1997). [https://doi.org/10.1016/S0304-3975\(96\)00260-5](https://doi.org/10.1016/S0304-3975(96)00260-5)
- Véronique Bruyère, Georges Hansel, Christian Michaux, and Roger Villemaire. 1994. Logic and p -recognizable sets of integers. *Bull. Belg. Math. Soc. Simon Stevin* (1994). <https://doi.org/10.36045/bbms/1103408547>
- Daniel Bundala and Joël Ouaknine. 2017. On parametric timed automata and one-counter machines. *Inf. Comput.* (2017). <https://doi.org/10.1016/J.IC.2016.07.011>
- Richard J. Büchi. 1960. Weak second-order arithmetic and finite automata. *Math. Logic Quart.* (1960). <https://doi.org/10.1002/malq.19600060105>
- Loris D'Antoni and Margus Veanes. 2017. Monadic second-order logic on finite sequences. In *POPL*. ACM, 232–245. <https://doi.org/10.1145/3009837.3009844>
- Loris D'Antoni and Margus Veanes. 2021. Automata modulo theories. *Commun. ACM* (2021). <https://doi.org/10.1145/3419404>
- Leonardo de Moura and Nikolaj Bjørner. 2008. Z3: An Efficient SMT Solver. In *TACAS*. https://doi.org/10.1007/978-3-540-78800-3_24
- Jacob Elgaard, Nils Klarlund, and Anders Møller. 1998. MONA 1.x: New Techniques for WS1S and WS2S. In *CAV (Lecture Notes in Computer Science)*. Springer, 516–520. <https://doi.org/10.1007/BFB0028773>
- Javier Esparza and Michael Blondin. 2023. *Automata Theory: An Algorithmic Approach*. The MIT Press.
- Tomás Fiedor, Lukáš Holík, Petr Janku, Ondrej Lengál, and Tomás Vojnar. 2017. Lazy Automata Techniques for WS1S. In *TACAS (Lecture Notes in Computer Science)*. 407–425. https://doi.org/10.1007/978-3-662-54577-5_24
- Tomás Fiedor, Lukáš Holík, Ondrej Lengál, and Tomás Vojnar. 2019. Nested antichains for WS1S. *Acta Informatica* 56, 3 (2019), 205–228. <https://doi.org/10.1007/S00236-018-0331-Z>
- Vijay Ganesh, Sergey Berezin, and David L. Dill. 2002. Deciding Presburger Arithmetic by Model Checking and Comparisons with Other Methods. In *FMCAD*. https://doi.org/10.1007/3-540-36126-X_11
- Seymour Ginsburg and Edwin H. Spanier. 1966. Semigroups, Presburger formulas, and languages. *Pacific J. Math.* 16, 2 (1966), 285–296. <https://doi.org/10.1145/2858784>
- Erich Grädel. 2020. Automatic Structures: Twenty Years Later. In *LICS*. <https://doi.org/10.1145/3373718.3394734>
- Florent Guépin, Christoph Haase, and James Worrell. 2019. On the Existential Theories of Büchi Arithmetic and Linear p -adic Fields. In *LICS*. <https://doi.org/10.1109/LICS.2019.8785681>
- Christoph Haase. 2012. *On the complexity of model checking counter automata*. Ph.D. Dissertation. University of Oxford, UK. <http://ora.ox.ac.uk/objects/uuid:f43bf043-de93-4b5c-826f-88f1bd4c191d>
- Christoph Haase and Alessio Mansutti. 2021. On Deciding Linear Arithmetic Constraints Over p -adic Integers for All Primes. <https://doi.org/10.4230/LIPICS.MFCS.2021.55>
- Christoph Haase and Jakub Różycki. 2021. On the expressiveness of Büchi arithmetic. In *FoSSaCS*. https://doi.org/10.1007/978-3-030-71995-1_16
- Peter Habermehl, Vojtěch Havlena, Michal Hečko, Lukáš Holík, and Ondřej Lengál. 2024. Algebraic Reasoning Meets Automata in Solving Linear Integer Arithmetic. In *CAV*. https://doi.org/10.1007/978-3-031-65627-9_3

- Philipp Hieronymi, Dun Ma, Reed Oei, Luke Schaeffer, Christian Schulz, and Jeffrey O. Shallit. 2024. Decidability for Sturmian words. *Log. Methods Comput. Sci.* (2024). [https://doi.org/10.46298/LMCS-20\(3:12\)2024](https://doi.org/10.46298/LMCS-20(3:12)2024)
- Philipp Hieronymi and Alonza Jr. Terry. 2018. Ostrowski Numeration Systems, Addition, and Finite Automata. *Notre Dame J. Formal Log.* (2018). <https://doi.org/10.1215/00294527-2017-0027>
- Bernard R. Hodgson. 1976. Théories décidables par automate fini. (1976). Ph.D. Thesis. Université de Montréal, Canada.
- J. P. Jones and Y. V. Matijasevič. 1984. Register Machine Proof of the Theorem on Exponential Diophantine Representation of Enumerable Sets. *J. Symb. Log.* 49, 3 (1984), 818–829. <https://doi.org/10.2307/2274135>
- Neil D. Jones. 1975. Space-bounded reducibility among combinatorial problems. *J. Comput. System Sci.* 11, 1 (1975), 68–85. [https://doi.org/10.1016/S0022-0000\(75\)80050-X](https://doi.org/10.1016/S0022-0000(75)80050-X)
- Bakhadyr Khoussainov and Anil Nerode. 1995. Automatic presentations of structures. In *Logic and Computational Complexity*. https://doi.org/10.1007/3-540-60178-3_93
- Dexter Kozen. 1997. *Automata and computability*. Springer.
- Anthony W. Lin and Rupak Majumdar. 2021. Quadratic Word Equations with Length Constraints, Counter Systems, and Presburger Arithmetic with Divisibility. *Log. Methods Comput. Sci.* (2021). [https://doi.org/10.46298/LMCS-17\(4:4\)2021](https://doi.org/10.46298/LMCS-17(4:4)2021)
- L. Lipshitz. 1978. The Diophantine Problem for Addition and Divisibility. *Trans. Amer. Math. Soc.* 235 (1978), 271–283.
- Kenneth Manders and Leonard Adleman. 1976. NP-complete decision problems for quadratic polynomials. In *STOC*. <https://doi.org/10.1145/800113.803627>
- Alessio Mansutti and Mikhail R. Starchak. 2025. One-Parametric Presburger Arithmetic Has Quantifier Elimination. <https://doi.org/10.4230/LIPICs.MFCS.2025.72>
- Albert R. Meyer. 1975. Weak monadic second order theory of successor is not elementary-recursive. In *Logic Colloquium*, Rohit Parikh (Ed.). Springer Berlin Heidelberg, Berlin, Heidelberg, 132–154.
- Marvin L. Minsky. 1967. *Computation: finite and infinite machines*. Prentice-Hall, Inc., USA.
- Hamoon Mousavi. 2016. Automatic Theorem Proving in Walnut. *CoRR* (2016). <http://arxiv.org/abs/1603.06017>
- John Nicol and Markus Frohme. 2026. Deconstructing Subset Construction - Reducing While Determinizing. In *TACAS*. https://doi.org/10.1007/978-3-032-22749-2_20
- Reed Oei, Dun Ma, Christian Schulz, and Philipp Hieronymi. 2021. Pecan: An Automated Theorem Prover for Automatic Sequences using Büchi Automata. *CoRR* (2021). <https://arxiv.org/abs/2102.01727>
- Derek C. Oppen. 1973. Elementary bounds for presburger arithmetic. In *Proceedings of the Fifth Annual ACM Symposium on Theory of Computing* (Austin, Texas, USA) (STOC '73). Association for Computing Machinery, New York, NY, USA, 34–37. <https://doi.org/10.1145/800125.804033>
- João Madeira Pereira, Filipe Marques, Pedro Adão, Hichem Rami Ait-El-Hara, Léo Andrès, Arthur Carcano, Pierre Chambart, Petar Maksimović, Nuno Santos, and José Fragoso" Santos. 2026. Smt.ml: A Multi-Backend Frontend for SMT Solvers in OCaml. In *TACAS*. https://doi.org/10.1007/978-3-032-22752-2_2
- Guillermo A. Pérez and Ritam Raha. 2022. Revisiting Parameter Synthesis for One-Counter Automata. In *30th EACSL Annual Conference on Computer Science Logic, CSL 2022, Göttingen, Germany (Virtual Conference), February 14-19, 2022*. <https://doi.org/10.4230/LIPICs.CSL.2022.33>
- Pierre Popoli, Jeffrey O. Shallit, and Manon Stipulanti. 2024. Additive Word Complexity and Walnut. In *FSTTCS*. <https://doi.org/10.4230/LIPICs.FSTTCS.2024.32>
- Mojżesz Presburger. 1929. Über die Vollständigkeit eines gewissen Systems der Arithmetik ganzer Zahlen. *Comptes-Rendus du ler Congrès des Mathématiciens des Pays Slaves* (1929), 92–101.
- Sylvain Schmitz. 2016. Complexity Hierarchies beyond Elementary. *ACM Trans. Comput. Theory* 8, 1, Article 3 (Feb. 2016), 36 pages. <https://doi.org/10.1145/2858784>
- Tobias Schuele and Klaus Schneider. 2006. Verification of data paths using unbounded integers: automata strike back. In *HVC*. https://doi.org/10.1007/978-3-540-70889-6_5
- Jeffrey O. Shallit. 1994. Numeration Systems, Linear Recurrences, and Regular Sets. *Inf. Comput.* (1994). <https://doi.org/10.1006/INCO.1994.1076>
- Jeffrey O. Shallit. 2022. *The Logical Approach to Automatic Sequences: Exploring Combinatorics on Words with WALNUT*. Cambridge University Press. <https://doi.org/10.1017/9781108775267>
- Mikhail R. Starchak. 2023. On the Existential Arithmetics with Addition and Bitwise Minimum. In *FoSSaCS*. https://doi.org/10.1007/978-3-031-30829-1_9
- Mikhail R. Starchak. 2024. Existential Definability of Unary Predicates in Büchi Arithmetic. In *CIE*. https://doi.org/10.1007/978-3-031-64309-5_18
- Roger Villemaire. 1992. The theory of $\langle \mathbb{N}; +, V_k, V_l \rangle$ is undecidable. *Theor. Comput. Sci.* 106, 2 (1992), 337–349. [https://doi.org/10.1016/0304-3975\(92\)90256-F](https://doi.org/10.1016/0304-3975(92)90256-F)
- Pierre Wolper and Bernard Boigelot. 1995. An Automata-Theoretic Approach to Presburger Arithmetic Constraints (Extended Abstract). In *SAS*. https://doi.org/10.1007/3-540-60360-3_30

Pierre Wolper and Bernard Boigelot. 2000. On the Construction of Automata from Linear Arithmetic Constraints. In *TACAS*.
https://doi.org/10.1007/3-540-46419-0_1

A PROOFS OF STATEMENTS IN SECTION 3.2

We recall the statements:

LEMMA 3.2. *Algorithm 1 respects its specification.*

PROOF. To see that $\mathcal{A}'$ is a valid NSA equivalent to $\mathcal{A}$ for all $k \in \mathbb{N}_{\geq 2}$, one may equivalently show that $\mathcal{A}'$ accepts all encodings of tuples for which $\mathcal{A}$ accepts some encoding of that tuple, and no more. Let us refer to functions in $[X \rightarrow \{0, k-1\}]$ as *sign tuples*. We say that a sign tuple s is *correct* for a word $\mathbf{w} = w_1 \dots w_0 \in \mathcal{M}_k$ whenever $s = w_l$.

Let $X = \{x_1, \dots, x_m\}$. Suppose that $m \in \mathcal{L}_{\text{msdf}}(\mathcal{A})(k)$, so there is some encoding $\mathbf{w} \in \mathcal{M}_k$ of m accepted by $\mathcal{A}$ with $t = k$. Let $\mathbf{w}' \in \mathcal{M}_k$ be the longest suffix of $\mathbf{w}$ which is not a valid encoding of m , and denote $\mathbf{w} = \mathbf{p}\mathbf{w}'$, where $\mathbf{p}$ is nonempty and consists only of correct sign tuples. Fix a run of $\mathcal{A}$ on $\mathbf{w}$. We take $q_0 q_1 \dots q_N$ to be the sequence of states encountered when reading $\mathbf{p}$ and immediately after, so $N = |\mathbf{p}| + 1$. While it contains cycles of states of form $q \dots q$, we cut out these cycles, replacing them with just q , until we reach a sequence $q'_0 q'_1 \dots q'_n$ of pairwise distinct states. Take any encoding $\mathbf{w}'' \in \mathcal{M}_k$ such that $\langle \mathbf{w}'' \rangle_k = \langle \mathbf{w} \rangle_k = m$, and let $\mathbf{w}'' = \mathbf{p}'\mathbf{w}'$. Then $\mathcal{A}'$ has a run where it spins on a fresh state $q'' \in Q'$ added in line 5, for all but the last symbol in $\mathbf{p}'$, then transitions to q'_n using the last one. Therefore, $m \in \mathcal{L}_{\text{msdf}}(\mathcal{A}')(k)$.

To see that nothing outside the language gets accepted, suppose $m \in \mathcal{L}_{\text{msdf}}(\mathcal{A}')(k)$ and let $\mathbf{w} \in \mathcal{M}_k$ be any encoding of m . The only outgoing transitions from q'_0 are added in line 7 of Algorithm 1. Take any accepting run of $\mathcal{A}'$ on $\mathbf{w}$. It must at some point reach some q_n , for some path $q_0 q_1 \dots q_n$, by consuming only correct sign tuples. By the condition on formula ψ , the run $q_0 q_1 \dots q_n$ is valid in $\mathcal{A}$, and consumes only correct sign tuples. Because from q_n onwards, acceptance of $\mathcal{A}$ is the same as of $\mathcal{A}'$, we obtain an accepting run of $\mathcal{A}$ on a word possibly differing by the amount of correct sign tuples, therefore encoding the same m . $\square$

LEMMA 3.3. *There are algorithms satisfying the following specifications for automata in $\mathcal{Aut}(\text{PAT})$:*

Determinization: *For a valid NSA $\mathcal{A}$, returns a valid DSA $\mathcal{A}'$ such that $\mathcal{L}_{\text{msdf}}(\mathcal{A}') = \mathcal{L}_{\text{msdf}}(\mathcal{A})$.*

Intersection: *For valid NSAs $\mathcal{A}_1$ and $\mathcal{A}_2$, return a valid NSA $\mathcal{A}'$ such that for every $k \in \mathbb{N}_{\geq 2}$, $\mathcal{L}_{\text{msdf}}(\mathcal{A}')(k) = \mathcal{L}_{\text{msdf}}(\mathcal{A}_1)(k) \cap \mathcal{L}_{\text{msdf}}(\mathcal{A}_2)(k)$.*

Complementation: *For a valid NSA $\mathcal{A}$, returns a valid NSA $\mathcal{A}'$ such that for every $k \in \mathbb{N}_{\geq 2}$, $\mathcal{L}_{\text{msdf}}(\mathcal{A}')(k) = \mathcal{M}_k \setminus \mathcal{L}_{\text{msdf}}(\mathcal{A})(k)$.*

PROOF OF DETERMINIZATION. Suppose that $\mathcal{A} = (\Phi, Q, q_0, F, \delta)$ is an NSA. To find an equivalent DSA, we will use the powerset construction, with special handling for transitions. For a set of formulae Φ , by $\text{BC}(\Phi)$ denote the set of Boolean combinations of Φ , which is infinite when Φ is nonempty. Namely, let $\mathcal{A}' = (\Phi', Q', q'_0, F', \delta')$, where

- $\Phi' := \delta'(Q' \times Q') \subseteq_{\text{fin}} \text{BC}(\Phi)$,
- $Q' := \mathcal{P}(Q)$,
- $q'_0 := \{q_0\}$,
- $F' := \{A \subseteq Q \mid A \cap F \neq \emptyset\}$, and
- $\delta'(A, B) := \bigwedge_{q' \in Q} \left(q' \in B \leftrightarrow \bigvee_{q \in A} \delta(q, q') \right)$.

Note that, since B is known, the $\leftrightarrow$ operator used in the definition of δ' can be elided, yielding a formula in $\mathcal{F}_{\text{PAT}}$. The elements Q' , q'_0 and F' are exactly the same as in the powerset construction for finite automata. The function δ' follows that semantic by establishing the conditions when the set $A \subseteq Q$ of states transitions exactly to the set $B \subseteq Q$ of states. It must be the case that B is exactly the set of states, to which at least one state in A transitions under the currently read symbol.

To prove that $\mathcal{A}'$ is indeed deterministic, we proceed by induction on run length. The inductive thesis is that $\mathcal{A}'$ has exactly one run for any word. For the empty word, this is trivially true. To see

the inductive step, suppose that $\mathcal{A}'$ is currently in state $A \in Q'$, and the symbol $a \in \sigma$ is currently being read. To see that there is at least one state, note that, for each state q' , each term of form $\bigvee_{q \in A} \delta(q, q')$ is either true or false for a . We can pick a subset $B \subseteq Q$ such that $q' \in B$ exactly when $a \models_k \bigvee_{q \in A} \delta(q, q')$.

To see that there is at most one state, let $B_1, B_2 \in Q'$ be two states it transitions into. Therefore, the current symbol $a \in \sigma$ satisfies both $\delta'(A, B_1)$ and $\delta'(A, B_2)$. We can equivalently express this as a conjunction

$$a \models_k \delta'(A, B_1) \wedge \delta'(A, B_2).$$

Because conjunction is associative and commutative, we can substitute the definition of δ' and rearrange to obtain

$$a \models_k \bigwedge_{q' \in Q} \left[\left(q' \in B_1 \leftrightarrow \bigvee_{q \in A} \delta(q, q') \right) \wedge \left(q' \in B_2 \leftrightarrow \bigvee_{q \in A} \delta(q, q') \right) \right].$$

We can then decompose this conjunction, obtaining, for every $q' \in Q$,

$$a \models_k \left(q' \in B_1 \leftrightarrow \bigvee_{q \in A} \delta(q, q') \right) \wedge \left(q' \in B_2 \leftrightarrow \bigvee_{q \in A} \delta(q, q') \right).$$

It follows that

$$a \models_k (q' \in B_1 \leftrightarrow q' \in B_2),$$

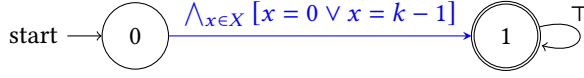
and therefore the sets B_1 and B_2 are equal by extensionality.

The language equivalence of $\mathcal{A}$ and $\mathcal{A}'$ follows by another induction on run length. The inductive thesis is as follows: a run of $\mathcal{A}$ on a word to state $q \in Q$ exists if and only if the only run of $\mathcal{A}'$ on that word ends up in a state A that contains the state q when understood as a subset of Q . It is easy to see that for the empty word this is true. Suppose $\mathcal{A}'$ is in the state $A \in Q'$ and transitions to $B \in Q'$ by reading $a \in \sigma$ after having previously read $w \in \sigma^*$. By the inductive assumption, A is exactly the set of states which runs on the word without the currently read symbol can reach. The very definition of δ' requires that the set of states after a transition is exactly the set of states that have a transition from at least one of states in A . Because every run over wa to q splits into a run over w to some q' and a transition from q' to q over a , the inductive step is established. From the inductive thesis and the definition of F' , $\mathcal{A}'$ accepts a word exactly when $\mathcal{A}$ does. Validity of $\mathcal{A}'$ follows from validity of $\mathcal{A}$, since these automata accept the same *encodings*, not just number tuples. $\square$

PROOF OF INTERSECTION. To establish intersection, we again proceed analogously to finite automata, this time using the product construction. Given $\mathcal{A}_1$ and $\mathcal{A}_2$, where $\mathcal{A}_i = (\Phi_i, Q_i, q_0^i, F_i, \delta_i)$, we construct $\mathcal{A}' = (\Phi', Q', q_0', F', \delta')$ as

- $\Phi' := \Phi_1 \wedge \Phi_2$,
- $Q' := Q_1 \times Q_2$,
- $q_0' := \langle q_0^1, q_0^2 \rangle \in Q'$,
- $F' := F_1 \times F_2 \subseteq Q'$, and
- $\delta'(\langle q_1, q_2 \rangle, \langle q_1', q_2' \rangle) := \delta_1(q_1, q_1') \wedge \delta_2(q_2, q_2')$.

The correctness of this construction again relies on induction on run length. This time, the inductive thesis is simpler: a run of $\mathcal{A}'$ to the state $\langle q_1, q_2 \rangle$ exists if and only if a run of $\mathcal{A}_1$ to the state q_1 and a run of $\mathcal{A}_2$ to the state q_2 both exist. The base case is immediate. Similarly, a run can be extended by a letter if and only if the runs of the two automata can be extended by that letter, by the definition of δ' . By the inductive thesis and the definition of F' , $\mathcal{A}'$ accepts a word exactly

Fig. 5. NSA for the language $\mathcal{M}_k$.

when both $\mathcal{A}_1$ and $\mathcal{A}_2$ do. Validity of $\mathcal{A}'$ follows from the validity of $\mathcal{A}_1$ and $\mathcal{A}_2$. Again, as for determinization, the intersection property holds for sets of *encodings*, not just numbers. Since the languages of both $\mathcal{A}_1$ and $\mathcal{A}_2$ are exact unions of sets of encodings of number tuples, so is their intersection. $\square$

PROOF OF COMPLEMENT. Recall that $\mathcal{M}_k$, defined in Section 2.5, is the language of valid encodings of number tuples. First, note that if, for some L , $\mathcal{M}_k \subseteq L$, then

$$\mathcal{M}_k \setminus \mathcal{L}_{\text{msdf}}(\mathcal{A})(k) = \mathcal{M}_k \cap (L \setminus \mathcal{L}_{\text{msdf}}(\mathcal{A})(k)).$$

Let $\mathcal{B} = (\Phi_{\mathcal{B}}, Q_{\mathcal{B}}, q_0^{\mathcal{B}}, F_{\mathcal{B}}, \delta_{\mathcal{B}})$ be a DSA recognizing the same language as $\mathcal{A}$. Now let $\mathcal{B}' := (\Phi_{\mathcal{B}}, Q_{\mathcal{B}}, q_0^{\mathcal{B}}, Q_{\mathcal{B}} \setminus F_{\mathcal{B}}, \delta_{\mathcal{B}})$ and $L = [X \rightarrow [0..k-1]]^*$. By determinicity of $\mathcal{B}$, it follows that

$$\mathcal{L}_{\text{msdf}}(\mathcal{B}')(k) = L \setminus \mathcal{L}_{\text{msdf}}(\mathcal{B})(k) = L \setminus \mathcal{L}_{\text{msdf}}(\mathcal{A})(k).$$

The automaton $\mathcal{B}'$ is also a DSA, therefore an NSA. Let $\mathcal{C}$ be the NSA recognizing the language $\mathcal{M}_k$, shown in Figure 5. We can take the automaton recognizing the intersection of the languages recognized by $\mathcal{B}'$ and $\mathcal{C}$ as the requested NSA $\mathcal{A}'$. $\square$

LEMMA 3.4. *Consider $\mathcal{A} \in \text{Aut}(PA\text{t})$ and $x \in X$. Then, for every $k \in \mathbb{N}_{\geq 2}$ and $m \in [X \rightarrow \mathbb{Z}]$, $m \in \mathcal{L}_{\text{msdf}}(\exists_x(\mathcal{A}))(k)$ if and only if there is $j \in \mathbb{Z}$ such that $m[x \mapsto j] \in \mathcal{L}_{\text{msdf}}(\mathcal{A})(k)$.*

PROOF. Suppose that $m \in \mathcal{L}_{\text{msdf}}(\exists_x(\mathcal{A}))(k)$. Therefore, there exists some encoding w of m accepted by $\exists_x(\mathcal{A})$. Fix an accepting run of the automaton. For each $i \in \{1, \dots, |w|\}$, we can look at the acceptance condition on the transition taken when reading w_i . There is some digit j_i found by the existential quantifier. Let $w'_i = w_i[x \mapsto j_i]$. Then $w' = w'_1 \dots w'_{|w|}$ is a word accepted by $\mathcal{A}$. This word encodes the tuple $m[x \mapsto j]$, where $j = \langle j_1 \dots j_{|w|} \rangle_k$.

Suppose that $m[x \mapsto j] \in \mathcal{L}_{\text{msdf}}(\mathcal{A})(k)$. We can fix an accepting run of $\mathcal{A}$ over some encoding of $m[x \mapsto j]$. A run of $\exists_x(\mathcal{A})$ of the same length could read the letters of this encoding without x , and guess the value of the appropriate digit to be the corresponding digit of j . Therefore $m \in \mathcal{L}_{\text{msdf}}(\exists_x(\mathcal{A}))(k)$. $\square$

LEMMA 3.5. *For every formula φ in $\mathcal{F}_{PBA}$ there is a NSA $\mathcal{A} \in \text{Aut}(PA\text{t})$ with $\mathcal{L}_{\text{msdf}}(\mathcal{A}) = \llbracket \varphi \rrbracket$.*

PROOF. The proof proceeds by structural induction on φ . First, as described in Section 3.2, one can represent any atomic formula with a formula only containing atoms of the form:

- $x = 0$,
- $x = 1$,
- $x \leq y$,
- $V_t(x, y)$, and
- $x + y = z$.

This is at the expense of adding new quantified variables. However, because the newly added variables are obtained using functional relations, one may introduce them using either existential or universal quantifier (i.e. they are definitional quantifiers), as not to increase the alternation rank of the formula. For example, the formula $x \leq 2$ is equivalent to

$$\exists y \exists z: (x \leq z) \wedge (y + y = z) \wedge (y = 1),$$

as well as

$$\forall y \forall z: [(y + y = z) \wedge (y = 1)] \rightarrow (x \leq z).$$

With that in mind, we can proceed with the induction. The base cases have already been outlined above, so we can proceed with the inductive steps. Note that the base cases are all valid NSA, and the induction shall preserve that property.

- For a formula of form $\varphi \wedge \psi$, let $\mathcal{A}_1$ and $\mathcal{A}_2$ be NSA such that, by the inductive assumption, $\mathcal{L}_{\text{msdf}}(\mathcal{A}_1) = \llbracket \varphi \rrbracket$ and $\mathcal{L}_{\text{msdf}}(\mathcal{A}_2) = \llbracket \psi \rrbracket$. By Lemma 3.3, we can construct a valid intersection NSA $\mathcal{A}'$ such that

$$\mathcal{L}_{\text{msdf}}(\mathcal{A}') = \mathcal{L}_{\text{msdf}}(\mathcal{A}_1) \cap \mathcal{L}_{\text{msdf}}(\mathcal{A}_2) = \llbracket \varphi \rrbracket \cap \llbracket \psi \rrbracket = \llbracket \varphi \wedge \psi \rrbracket.$$

- For a formula of form $\neg \varphi$, let $\mathcal{A}$ be the NSA such that $\mathcal{L}_{\text{msdf}}(\mathcal{A}) = \llbracket \varphi \rrbracket$. By Lemma 3.3, we can construct a valid complement NSA $\mathcal{A}'$. Then

$$\mathcal{L}_{\text{msdf}}(\mathcal{A}') = \mathcal{M}_k \setminus \mathcal{L}_{\text{msdf}}(\mathcal{A}) = \mathcal{M}_k \setminus \llbracket \varphi \rrbracket = \llbracket \neg \varphi \rrbracket.$$

- For a formula of form $\varphi \vee \psi$, use De Morgan's Laws,

$$\varphi \vee \psi \equiv \neg [(\neg \varphi) \wedge (\neg \psi)].$$

- For a formula of form $\exists x: \varphi$, let $\mathcal{A}$ be the NSA such that $\mathcal{L}_{\text{msdf}}(\mathcal{A}) = \llbracket \varphi \rrbracket$. By Lemma 3.4, we can construct a projection NSA $\mathcal{A}'$ with respect to the variable x . Let $\mathcal{A}''$ be a valid NSA recognizing all valid msdf encodings of the number tuples that have any encoding recognized by $\mathcal{A}'$, constructed using Algorithm 1. Then, for every $k \in \mathbb{N}_{\geq 2}$,

$$\mathcal{L}_{\text{msdf}}(\mathcal{A}'')(k) = \pi_x(\mathcal{L}_{\text{msdf}}(\mathcal{A})(k)) = \pi_x(\llbracket \varphi \rrbracket_k) = \llbracket \exists x: \varphi \rrbracket_k,$$

therefore

$$\mathcal{L}_{\text{msdf}}(\mathcal{A}'') = \llbracket \exists x: \varphi \rrbracket.$$

- For a formula of form $\forall x: \varphi$, use De Morgan's Laws,

$$\forall x: \varphi \equiv \neg \exists x: \neg \varphi.$$

This concludes the proof, and establishes the effectiveness of the procedure. $\square$

PROPOSITION A.1. *The running time of this procedure can be bounded from above by a function related to TOWER(n), defined as*

$$\text{TOWER}(n) = \underbrace{2^{2^{\dots^2}}}_{n \text{ copies of } 2},$$

with n being the representation length of the input formula.

For a formula in prenex form, this running time is strictly related to the alternation rank. Restricted to formulae with a fixed number of alternations, this procedure works in elementary time.

However, there is no substantially faster procedure for such conversion. In particular, it cannot be done in elementary time for arbitrary formulae.

PROOF. The upper bound follows due to determinizations induced by complementation. Each complementation blows the size of the automaton up exponentially, and the formula can have an unbounded number of them, even in the form of quantifier alternations.

For the lower bound, suppose there is some procedure P that turns formulae of PBA into NSA. Let us take any sentence φ of 2BA. We replace all occurrences of V_2 (of 2BA) with V_t (of PBA) in the sentence φ , obtaining $\varphi' \in \mathcal{F}_{\text{PBA}}$. We run P on the sentence $t = 2 \wedge \varphi'$, obtaining some NSA $\mathcal{A}$. Next, we make the label formulae quantifier-free by Fact 3.1, which can be done in triply-exponential time by [Oppen 1973]. We remove all edges that are false for $t = 2$, and erase the labels of those

that are true, obtaining an unlabelled NFA $\mathcal{A}'$. This can be done in polynomial time. Language nonemptiness of $\mathcal{L}(\mathcal{A}')$ is equivalent to the truth of φ . Checking if an NFA accepts any word is equivalent to checking reachability in a directed graph, which is decidable in nondeterministic logarithmic space due to [Jones 1975, Theorem 21], and therefore in polynomial time.

However, the theory of 2BA is known to be TOWER-hard [Bruyère et al. 1994; Meyer 1975], for the class TOWER as understood in [Schmitz 2016, Section 2.3]. Substitution of V_2 , quantifier elimination, substitution of $t = 2$ and the subsequent NFA nonemptiness check are all computable in elementary time. Therefore, a correct procedure P cannot have its worst-case running time bounded by an elementary function of n , or significantly better than TOWER(n) at all. $\square$

B PROOFS OF STATEMENTS IN SECTION 3.4

LEMMA 3.10. *Algorithm 2 respects its specification and terminates.*

PROOF OF CORRECTNESS. We will show that the following are equivalent for a base $k \in \mathbb{N}_{k \geq 2}$:

- (1) φ is satisfiable for $t = k$,
- (2) $\mathcal{L}_{\text{msdf}}(\mathcal{A})(k)$ is nonempty,
- (3) there exists a sequence $r_0, \dots, r_n \in Q$ (including the newly created state $\tilde{q}_0$) such that
 - $r_0 = \tilde{q}_0$,
 - $r_n \in F$, and
 - for each $i \in [1..n]$, $k \in b(r_{i-1}, r_i)$;
- (4) $k \in m(\tilde{q}_0)$ after the algorithm stabilizes.

(1) $\iff$ (2): Follows directly from Lemma 3.5. Note that the addition of $\tilde{q}_0$ and subsequent change in transitions does not change the acceptance for valid encodings.

(2) $\implies$ (3): Suppose that $\langle w_l \dots w_0 \rangle_k \in \mathcal{L}_{\text{msdf}}(\mathcal{A})(k)$. Fix an accepting run over w . Let $n := l + 1$, and let r_i for $i \in [0..n]$ be the respective states of the accepting run after reading i symbols. Because it is an accepting run, it must satisfy the first two conditions. Furthermore, the transition from r_{i-1} to r_i for $i \in [1..n]$ can be taken for w_{i-1} and therefore is satisfiable, so $k \in b(r_{i-1}, r_i)$.

(3) $\implies$ (2): We know that, for all $i \in [1..n]$, $k \in b(r_{i-1}, r_i)$, so $\delta(r_{i-1}, r_i)$ is satisfiable for $t = k$. Let $l = n - 1$. We choose, for each $i \in [1..n]$, a symbol w_{n-i} in $[0..k - 1]^m$ that satisfies $\delta(r_{i-1}, r_i)$. The initial transformation of $\mathcal{A}$ ensures that w_l is a valid sign tuple, and thus w encodes a valid number. Then $\langle w_l \dots w_0 \rangle_k \in \mathcal{L}_{\text{msdf}}(\mathcal{A})(k)$.

(3) $\implies$ (4): Because $r_n \in F$, initially $k \in m(r_n)$. While $k \in m(r_i)$, and $k \notin m(r_{i-1})$ for any $i \in [1..n]$, the algorithm has outstanding work, so it cannot stabilize. Then k becomes a member of $m(r_i)$ by the update rule. In Section 3.4, we show that the algorithm always stabilizes. Therefore, in the end, $k \in m(r_0) = m(\tilde{q}_0)$.

(4) $\implies$ (3): We build the path $r_0 \dots r_n$ inductively, starting from $r_0 := \tilde{q}_0$. We assume that $k \in m(r_i)$, and it was first the case after $l_i \in \mathbb{N}$ iterations. If $l_i = 0$, it means that $r_i \in F$, and we set $n := i$. Note that this particular case can only happen for $i > 0$. Otherwise, because it was introduced in iteration $l_i > 0$, it must be the case that $k \in m(r_{i+1})$ and $k \in b(r_i, r_{i+1})$ for some state which we can call r_{i+1} . Also, because k already was a member of $m(r_{i+1})$ at that point, $l_{i+1} < l_i$. We continue the construction in r_{i+1} . Because l_i strictly decreases and takes only natural values, this procedure must terminate for some n . The resulting sequence r_i satisfies the requirements of (3). $\square$

C PROOFS OF STATEMENTS IN SECTION 4

COROLLARY 4.2. *The EXISTENCE problem for the $\exists^*$ fragment of $\mathcal{F}_{\text{PBA}[t, \&_t]}$ is undecidable.*

PROOF. For a recursively enumerable set $S \subseteq \mathbb{N}$ of numbers accepted by a 3-counter machine $\mathcal{M}$, let $\varphi(x)$ be the existential formula from Theorem 4.1. We have $S = \emptyset$ if and only if $\mathcal{B}(\varphi) = \emptyset$. Hence, the non-emptiness problem for 3-counter machines reduces to the EXISTENCE problem for the existential fragment of $\mathcal{F}_{PBA}[t, \&_t]$. $\square$

D PROOFS OF STATEMENTS IN SECTION 5

LEMMA 5.1. *Let Q be the set defined in line 4 of Algorithm 3, with respect to the integers $a_1, \dots, a_n, b$ in line 2. Then, (1) $b \in Q$, and (2) $\left\lfloor \frac{v - \sum_{i=1}^n a_i \cdot w(x_i)}{k} \right\rfloor \in Q$ for all $v \in Q, k \in \mathbb{N}_{\geq 2}$ and $w: X \rightarrow [0..k-1]$.*

PROOF. The fact that b belongs to Q is straightforward (simply take $k = 1$ and $\ell = 0$ in line 4).

Let us now prove the second item. Take $v \in Q, k \in \mathbb{N}_{\geq 2}$ and $w: X \rightarrow [0..k-1]$, then, by definition of the set Q , we have $v = \left\lfloor \frac{b}{j} \right\rfloor + \ell$ for some $j \in [1..|b|+1]$ and $\ell \in [-a_+..-a_-]$ for the integers $a_+ := \sum_{a_i > 0} a_i$ and $a_- := \sum_{a_i < 0} a_i$.

We will now establish the inequality $\left\lfloor \frac{b}{j \cdot k} \right\rfloor - a_+ \leq \left\lfloor \frac{v - \sum_{i=1}^n a_i \cdot w(x_i)}{k} \right\rfloor \leq \left\lfloor \frac{b}{j \cdot k} \right\rfloor - a_-$, which implies that $\left\lfloor \frac{v - \sum_{i=1}^n a_i \cdot w(x_i)}{k} \right\rfloor \in Q$. Let us only prove the lower bound, because the upper bound is analogous:

$$\begin{aligned}
 \left\lfloor \frac{v - \sum_{i=1}^n a_i \cdot w(x_i)}{k} \right\rfloor &= \left\lfloor \frac{\left\lfloor \frac{b}{j} \right\rfloor + \ell - \sum_{i=1}^n a_i \cdot w(x_i)}{k} \right\rfloor && \text{(definition of } v \text{)} \\
 &= \left\lfloor \frac{\frac{b}{j} + \ell - \sum_{i=1}^n a_i \cdot w(x_i)}{k} \right\rfloor && \left(\left\lfloor \frac{\lfloor a \rfloor + b}{c} \right\rfloor = \left\lfloor \frac{a+b}{c} \right\rfloor \text{ for } a \in \mathbb{R}, b \in \mathbb{Z}, c \in \mathbb{N}_{\geq 1} \right) \\
 &\geq \left\lfloor \frac{\frac{b}{j} - a_+ - a_+ \cdot (k-1)}{k} \right\rfloor && (\ell \geq -a_+ \text{ and } \sum_{i=1}^n a_i \cdot w(x_i) \leq a_+ \cdot (k-1)) \\
 &= \left\lfloor \frac{b}{j \cdot k} - a_+ \right\rfloor = \left\lfloor \frac{b}{j \cdot k} \right\rfloor - a_+ && (\lfloor a + b \rfloor = \lfloor a \rfloor + b \text{ for } a \in \mathbb{R} \text{ and } b \in \mathbb{Z}) \quad \square
 \end{aligned}$$

LEMMA 5.2. *Algorithm 3 respects its specification.*

PROOF. Consider an inequality $\varphi := \sum_{i=1}^n a_i \cdot x_i \leq b$. Following lines 1 and 2 of Algorithm 3, we can assume that $\gcd(a_1, \dots, a_n) = 1$. Let $\mathcal{A} = (\Phi, Q \cup \{q_0\}, q_0, F, \delta)$ be the automaton for φ constructed by Algorithm 3. To prove the lemma, we are going to show that for every $k \in \mathbb{N}_{\geq 2}$ we have $\mathcal{L}_{\text{msdf}}(\mathcal{A})(k) = \llbracket \varphi \rrbracket_k$, and that $\mathcal{A}$ is a valid NSA. For this, it suffices to consider $k \in \mathbb{N}_{\geq 2}$, a word $\mathbf{w} = w_\ell \dots w_0 \in \mathcal{M}_k$, and $v \in Q \subseteq \mathbb{Z}$. By induction on ℓ , we show that $\sum_{i=1}^n a_i \cdot \llbracket \mathbf{w} \rrbracket_k(x_i) \leq v$ if and only if there is a path $q_0 \dots q_{\ell+1}$ such that $q_{\ell+1} = v$ and $w_{\ell+1-i} \models_k \delta(q_{i-1}, q_i)$ for every $i \in [1..\ell+1]$. As $b \in F$ (line 4), taking $v = b$ proves the lemma.

base case: $\ell = 0$. For the base case $\mathbf{w} = w_0$, it suffices to show that $w_0 \models_k \delta(q_0, v)$ if and only if $\sum_{i=1}^n a_i \cdot \llbracket w_0 \rrbracket_k(x_i) \leq v$. Recall that, for all $x \in X$, either $\llbracket w_0(x) \rrbracket_k = 0$ and $w_0(x) = 0$, or $\llbracket w_0(x) \rrbracket_k = -1$ and $w_0(x) = k-1$. In other words, $\llbracket w_0 \rrbracket_k(x) = \llbracket w_0(x) \rrbracket_k = \frac{w_0(x)}{1-k}$. Then,

$$\begin{aligned}
 w_0 \models_k \delta(q_0, v) &\Leftrightarrow \sum_{i=1}^n a_i \cdot w_0(x_i) \geq v \cdot (1-k) && \text{(definition of } \delta \text{ in line 5)} \\
 &\Leftrightarrow \sum_{i=1}^n a_i \cdot \frac{w_0(x_i)}{1-k} \leq v && \text{(divide by } 1-k < 0 \text{)} \\
 &\Leftrightarrow \sum_{i=1}^n a_i \cdot \llbracket w_0 \rrbracket_k(x_i) \leq v && \left(\llbracket w_0 \rrbracket_k(x) = \frac{w_0(x)}{1-k} \right)
 \end{aligned}$$

induction step: $\ell \geq 1$. Let $\mathbf{w}' = w_\ell \dots w_1$. So, $\llbracket \mathbf{w} \rrbracket_k(x) = \llbracket \mathbf{w}' \rrbracket_k(x) \cdot k + w_0(x)$ for all $x \in X$.

For the right-to-left direction, suppose there is a path $q_0 \dots q_{\ell+1}$ such that $q_{\ell+1} = v$ and $w_{\ell+1-i} \models_k \delta(q_{i-1}, q_i)$ for every $i \in [1..\ell+1]$. Note that q_ℓ is an integer from Q , and that

$w_0 \models_k \delta(q_\ell, q_{\ell+1})$ implies $q_\ell \cdot k \leq q_{\ell+1} - \sum_{i=1}^n a_i \cdot w_0(x_i)$, from the definition of δ in line 5. Therefore,

$$\begin{aligned} \sum_{i=1}^n a_i \cdot \langle \mathbf{w} \rangle(x_i)_k &= (\sum_{i=1}^n a_i \cdot \langle \mathbf{w}' \rangle_k(x_i)) \cdot k + (\sum_{i=1}^n a_i \cdot w_0(x_i)) \\ &\leq q_\ell \cdot k + (\sum_{i=1}^n a_i \cdot w_0(x_i)) && \text{(I.H.)} \\ &\leq q_{\ell+1} = v && \text{(by } w_0 \models_k \delta(q_\ell, q_{\ell+1})) \end{aligned}$$

For the left-to-right direction, suppose that $\sum_{i=1}^n a_i \cdot \langle \mathbf{w} \rangle_k(x_i) \leq v$. By definition of $\mathbf{w}'$ together with the fact that $\sum_{i=1}^n (a_i \cdot \langle \mathbf{w}' \rangle_k(x_i))$ is an integer, we have

$$(\sum_{i=1}^n a_i \cdot \langle \mathbf{w}' \rangle_k(x_i)) \leq \left\lfloor \frac{v - \sum_{i=1}^n a_i \cdot w_0(x_i)}{k} \right\rfloor.$$

Let $v' := \left\lfloor \frac{v - \sum_{i=1}^n a_i \cdot w_0(x_i)}{k} \right\rfloor$, which belongs to Q by Lemma 5.1. By induction hypothesis, there is a path $q_0 \dots q_\ell$ such that $q_\ell = v'$ and $w_{\ell+1-i} \models_k \delta(q_{i-1}, q_i)$ for every $i \in [1..\ell]$. By definition of v' and of the transition function δ , we get $w_0 \models_k \delta(v', v)$. So, this path can be extended to $q_0 \dots q_\ell q_{\ell+1}$ such that $q_{\ell+1} = v$ and $w_{\ell+1-i} \models_k \delta(q_{i-1}, q_i)$ for all $i \in [1..\ell+1]$. $\square$